

Advanced NLP (ANLP M26)

Comprehensive Endsem/Quiz Study Guide

Lectures: 30 July – 17 September 2026

Compiled from transcripts, slide decks, handwritten notes & PYQs

Contents

How to Use This Guide	2
1 Lecture 1 (30 July): Course Logistics & Linguistic Foundations	3
1.1 Overview	3
1.2 Linguistic Background (asked about repeatedly in PYQs indirectly via language modelling context)	3
1.3 Why Transformers Replace RNN/LSTM Pipelines	4
1.4 Course Structure Notes	4
1.5 Practice & Review	4
2 Lecture 2 (3 August): Introduction to the Transformer	6
2.1 Overview	6
2.2 Embeddings: A Brief Evolution	6
2.3 Why RNNs/LSTMs Fall Short	6
2.4 Self-Attention Core Mechanics	6
2.5 Multi-Head Attention	7
2.6 What Self-Attention Alone Is Missing	7
2.7 Positional Encoding (Preview – full treatment in Lecture 6)	7
2.8 Add & Norm (Residual Connections + Layer Normalization)	8
2.9 The Full Encoder–Decoder Diagram	8
2.10 Transformer Type Zoo	8
2.11 Parameter Counting (worked example, ~1B model)	8
2.12 Transformer Gains & Drawbacks (Summary)	9
2.13 Practice & Review	9
3 Lecture 3 (6 August): Self-Attention Projections & Why Dot-Product	11
3.1 Overview	11
3.2 The Space of Possible Relevance Functions	11
3.3 From Position/Embedding to Q, K via Linear Projection	11
3.4 Normalisation of Scores: Why Softmax	11
3.5 Context Vector and the Rest of the Layer	12
3.6 Practice & Review	12

4	Lecture 4 (10 August): The Transformer Design Cascade, Scaling & Efficiency Preview	14
4.1	Overview	14
4.2	Why Parallel Processing Wins (Recap + Formal Complexity)	14
4.3	Why Multiple Attention Layers (Depth)	14
4.4	Normalisation Placement Preview: Pre-Norm vs Post-Norm	14
4.5	Model Scaling & Parameter Budget Reasoning	14
4.6	Historical Model Timeline (as discussed)	15
4.7	Parameter-Efficient Fine-Tuning Preview: LoRA	15
4.8	Mixture-of-Experts (Preview – full treatment in Lecture 10)	15
4.9	Long-Context Challenges (Preview – full treatment in Lecture 7)	15
4.10	Practice & Review	15
5	Lecture 5 (13 August): Tokenization Deep Dive	17
5.1	Overview	17
5.2	What Is a Token?	17
5.3	The Vocabulary Explosion Problem	17
5.4	Zipf’s Law	17
5.5	Unicode, Scripts, and Bytes	17
5.6	Statistical Foundations: MLE and EM	18
5.7	Byte-Pair Encoding (BPE) – Step by Step	18
5.8	WordPiece vs. SentencePiece vs. Byte-level BPE	19
5.9	Limitations of Sub-word Tokenizers	19
5.10	Beyond Tokenization: The Byte Latent Transformer (mentioned 17/8 in handwritten notes)	19
5.11	Practice & Review	19
6	Lecture 6 (17 August): Attention as a Graph & Positional Encoding Variants	21
6.1	Overview	21
6.2	Attention as a Fully-Connected Graph	21
6.3	Word-Order Phenomena in Natural Language	21
6.4	Positional Encoding: Full Taxonomy	22
6.5	Causal Mask	22
6.6	Proximity Bias / Long-Distance Dependencies	23
6.7	Self-Attention, KV, and the Bridge into Efficient Attention (from the same slide deck)	23
6.8	Multi-Query Attention (MQA), Grouped-Query Attention (GQA), Multi-Head Latent Attention (MLA) – Introduced Here, Elaborated in Lecture 7	23
6.9	Longformer – Efficient Attention for Long Documents (also in this deck)	23
6.10	Transformer Models Over the Years (attention-mechanism mapping, from slides)	24
6.11	Practice & Review	24
7	Lecture 7 (20 August / “More Attention” 24 Aug deck): Efficient Attention, KV-Cache Engineering & Encoder-Decoder Wiring	26
7.1	Overview	26
7.2	Encoder-Decoder Data Flow (Recap + Detail)	26
7.3	The Quadratic Bottleneck, Formally	26
7.4	K/V Caching – The Full Mechanics	26
7.5	Reducing Attention’s Parameter/Memory Footprint: MQA, GQA, MLA – Full Detail	27

7.5.1	Multi-Query Attention (MQA, Shazeer 2019)	27
7.5.2	Grouped-Query Attention (GQA, Ainslie et al. 2023)	27
7.5.3	Multi-Head Latent Attention (MLA, DeepSeek-V2, Liu et al. 2024)	27
7.6	Parameter/Cache Comparison Table (worked, ~1B model)	27
7.7	Precision & Quantisation	28
7.8	Standard Attention – Restated Disadvantages (“More Attention” deck)	28
7.9	Sliding (Window) Attention – Formalised	28
7.10	Sparse Attention	28
7.11	Hybrid Attention (BigBird, Zaheer et al. 2020)	29
7.12	Linear Attention	29
7.13	Practice & Review	30
8	Lecture 8 (7 September): Normalization Fundamentals	32
8.1	Overview	32
8.2	Why Normalize: The Loss-Geometry Argument	32
8.3	Three Conceptual Spaces	32
8.4	Why Normalization Also Fights Over-fitting	33
8.5	Batch Normalization vs. Layer Normalization	33
8.6	BatchNorm vs. LayerNorm – Which Axis Is Normalised (slide-deck formalism)	34
8.7	HybridNorm (Zhuo et al., NeurIPS 2026) – a 2026 refinement	34
8.8	Practice & Review	34
9	Lecture 9 (10 September): Pre-Norm vs. Post-Norm, RMSNorm, Residuals & Representation Collapse	36
9.1	Overview	36
9.2	Residual (Skip) Connections	36
9.3	Post-Norm vs. Pre-Norm vs. DeepNorm	36
9.4	Representation Collapse	37
9.5	Emergent Features	37
9.6	Non-Linearity Recap: FFN and Beyond (Bridges to Lecture 10)	37
9.7	Practice & Review	37
10	Lecture 10 (17 September): Neural Network Foundations, Activation Functions, Gating & Mixture-of-Experts	40
10.1	Overview	40
10.2	Historical Timeline	40
10.3	Linear Separability, XOR, and VC-Dimension	40
10.4	Three Core Learning Paradigms	41
10.5	Normalization (compression of the error range) Recap in this Lecture	41
10.6	Activation Function Taxonomy	41
10.7	The Feed-Forward Network (FFN) – Formal Definition (FFNN slide deck)	42
10.8	Gating Mechanisms & GLU Variants (formal treatment)	42
10.9	Mixture-of-Experts (MoE) – Comprehensive Treatment	44
10.9.1	MoE Components	44
10.9.2	Routing Algorithms	45
10.9.3	Load Balancing	46
10.9.4	DeepSeekMoE – Shared + Routed Experts (Liu et al., DeepSeek-V2/V3)	46
10.10	Transformer Models Over the Years – 2026 Snapshot (params vs. active params)	47

10.11	Residual / Highway Networks Recap in This Lecture's Framing	47
10.12	Practice & Review	47
11	Bridging Notes: What Only the Handwritten Notes Reveal	51
12	Seminar 1: Data Creation, Curation & Evaluation for LLMs	52
12.1	Overview	52
12.2	What Makes Good (Generated) Data?	52
12.3	Why Not Just Use the Internet?	52
12.4	Curation Is Subtractive Design: The FineWeb Pipeline	53
12.5	From Curation to Synthesis: Two Stages of Generation	53
12.5.1	Synthesis #1 – Pretraining: WRAP (Web Rephrase Augmented Pretraining)	54
12.5.2	Synthesis #2 – Post-Training: Self-Instruct and Evol-Instruct	54
12.6	The Student Inherits the Teacher (and Model Collapse)	54
12.7	Evaluation: Benchmark Decontamination	55
12.8	Other Evaluation Methods (Short Notes)	56
12.9	Open Problems (flagged by the seminar as unresolved)	56
12.10	Practice & Review	56
13	Seminar 2: Reasoning in Language Models	59
13.1	Overview	59
13.2	Method 1: Chain of Thought (CoT) – Wei et al., NeurIPS 2022	59
13.3	Method 2: Supervised Fine-Tuning on Reasoning Traces – STaR (Zelikman et al., 2022)	60
13.4	Method 3: Reinforcement Learning (RL) – The Full Stack	60
13.4.1	REINFORCE (Williams, 1992)	61
13.4.2	TRPO (Schulman et al., ICML 2015)	62
13.4.3	PPO (Schulman et al., 2017)	62
13.4.4	RLHF (Ouyang et al., InstructGPT, 2022)	63
13.4.5	RLOO (Kool, van Hoof & Welling, ICLR 2019 workshop)	64
13.4.6	GRPO (Shao et al., DeepSeekMath, 2024)	64
13.5	Efficient Reasoning: Thinking Budgets	65
13.6	Recent Advances (Pointers)	65
13.7	Practice & Review	65
14	Previous-Year Question Patterns & Predictions	68
14.1	2023 Mid-Sem (M23CS7.501, Sep 22 2023) – 30 marks, 1.5 hrs	68
14.2	2024 Mid-Sem (M24CS7.501, Sep 23 2024) – 30 marks, 1.5 hrs	68
14.3	Cross-Year Pattern Recognition	68
15	Quick-Reference Formula Sheet	70
16	Rapid Self-Test Bank (Answer Key at End)	72

How to Use This Guide

This guide reconstructs all 10 class sessions held so far (30 July – 17 September 2026) *plus* the two student seminars on the syllabus (Data Creation; Reasoning in Language Models), synthesising the audio-transcript summaries, the official slide decks, your handwritten notes (30/7 – 17/9), and 2023/2024 previous-year mid-sem papers from the same instructor. Each lecture/seminar section has: an overview, key concepts, worked derivations, and a Practice & Review block. Section 14 analyses recurring question patterns from the instructor’s past papers. Section 15 is a one-stop formula sheet for last-minute revision. Section 16 is a rapid self-test bank.

Study priority (based on repeated emphasis across lectures and PYQ patterns):

(1) Self-attention mechanics & multi-head attention math, (2) Positional encodings (absolute, sinusoidal, relative, RoPE), (3) Tokenization (BPE, WordPiece, SentencePiece, Zipf’s law), (4) KV-cache and efficient attention (MQA, GQA, MLA), (5) Normalization (BatchNorm, LayerNorm, RMSNorm, Pre- vs Post-Norm), (6) Encoder-decoder architecture & cross-attention, (7) Neural net foundations (activation functions, gating, MoE), (8) RL for LLMs (PPO, RLHF, GRPO – if the seminar material is in scope for your assessment), (9) Data curation/synthesis pipeline (BPE-adjacent but distinct: FineWeb, WRAP, Self-Instruct, model collapse – if in scope).

1 Lecture 1 (30 July): Course Logistics & Linguistic Foundations

1.1 Overview

The opening lecture (slides: *01_ANLP-M26-Lec1-Logistics.pdf*) sets up the course as a deep dive into the architectures underlying modern NLP (principally Transformers), rather than an applications survey. An introductory NLP course is *not* a strict prerequisite, but background is expected. The course is capped near 200 students and is interaction-heavy.

Instructor's framing

The course studies **design choices and trade-offs** in NLP architectures, not just how to use them. Emphasis throughout the term is on *why* a component exists (engineering constraint) rather than treating it as handed-down fact.

1.2 Linguistic Background (asked about repeatedly in PYQs indirectly via language modelling context)

- **Language** is described as a system that emerges from friction between speakers and listeners — it evolves to convey meaning efficiently, shaped by context, shared knowledge, and inference.
- **Max Weinreich's aphorism** (in the handwritten notes): “*A language is a dialect with an army and navy.*” This captures that the language/dialect boundary is political and social, not purely linguistic.
- **Dialect** arises through *isolation* (populations diverge from a base language over time without new contact).
- **Pidgin**: a simplified contact language with no native speakers, built for basic communication between groups without a shared language (arises from *contact*).
- **Creole**: a pidgin that has *acquired native speakers* and stabilised into a full language with its own developed grammar and vocabulary; arises from continued contact or resolving conflict/coexistence pressures.
- **Code-switching** vs **code-mixing**: switching = alternating between two full languages across utterances/sentences; mixing = blending elements of multiple languages within a single utterance.

Quick distinctions

Pidgin → Creole: Pidgin (no native speakers, minimal grammar) $\xrightarrow{\text{nativisation}}$ Creole (native speakers, full grammar).

Dialect: same language family diverging via isolation, no contact needed.

Language vs Dialect: socio-political distinction (“army and navy”), not a clean linguistic one.

1.3 Why Transformers Replace RNN/LSTM Pipelines

The historical arc sketched: **Statistical MT** (e.g. old Google Translate) $\Rightarrow$ **Attention-augmented seq2seq (RNN/LSTM + attention)** $\Rightarrow$ **Transformers**.

- RNN/LSTM drawback: **latency** — sequential processing means token t must wait on token $t - 1$, so the model cannot be parallelised across the sequence dimension.
- Transformers are **parallelised**: since attention treats the input as a matrix, all pairwise token interactions can be computed as one big matrix multiplication, which GPUs execute efficiently.

1.4 Course Structure Notes

- Seminar component: 16 students present, starting around lecture 13, each with ~ 20 minutes (presentation + Q&A); seminar paper $\approx$ survey paper but with deeper critical analysis.
- Course project: propose a **novel architecture** (not just apply an existing one) – graded on creativity, technical soundness, and evaluation methodology.

1.5 Practice & Review

Q1. [MC] What is the primary architectural advantage of Transformers over RNNs for sequence modelling?

- (A) Fewer parameters
- (B) Parallelisable computation via matrix multiplication
- (C) No need for embeddings
- (D) Guaranteed convergence

Answer: (B). Self-attention expresses all pairwise token interactions as $QK^\top$, a single matrix multiply, removing the sequential dependency of RNNs.

Q2. [Short Answer] Distinguish Pidgin, Creole, and Dialect. *Answer:* Pidgin = simplified contact language, no native speakers, minimal grammar. Creole = a pidgin that gained native speakers and a fully developed grammar. Dialect = a variant of a language that diverged through isolation rather than contact; the language/dialect line is politically drawn (“a language is a dialect with an army and navy”).

Q3. [Short Answer] Why can RNN-based sequence-to-sequence models not be fully parallelised during training across the time dimension? *Answer:* Each hidden state h_t depends recursively on h_{t-1} , so the forward pass must proceed step by step; this sequential dependency prevents a single GPU kernel from processing all timesteps simultaneously, unlike self-attention’s all-pairs matrix multiplication.

Q4. [Essay] The instructor frames the evolution of NLP as Statistical $\rightarrow$ Attention $\rightarrow$ Transformers. Explain the limitation that motivated each transition. *Answer:* Statistical MT (phrase tables, alignment models) lacked the capacity to model long-range, context-dependent meaning and required heavy hand-engineering; neural seq2seq with attention fixed the fixed-length bottleneck of vanilla encoder-decoder RNNs by letting the decoder look back at all encoder

states, but retained RNNs' sequential latency and vanishing-gradient issues over long sequences; Transformers removed recurrence entirely, replacing it with self-attention computed in parallel across the whole sequence, solving both the long-range dependency problem and the parallelisation problem simultaneously.

2 Lecture 2 (3 August): Introduction to the Transformer

2.1 Overview

Slides: *02_ANLP-M26-Lec2-Transformers.pdf* (80 slides). This lecture builds from word embeddings up through the full encoder-decoder Transformer diagram, and articulates *why* each block exists.

2.2 Embeddings: A Brief Evolution

- **One-hot vectors** $\rightarrow$ **word2vec** / **GloVe** $\rightarrow$ **Transformer-based contextual embeddings**.
- Each step improves how much *context and semantic meaning* is captured; one-hot carries none, static embeddings (word2vec) carry one fixed vector per word type, contextual embeddings vary by sentence.

2.3 Why RNNs/LSTMs Fall Short

- **Vanishing gradients**: repeated multiplication of small Jacobians during backprop-through-time shrinks gradients exponentially over long sequences, so early tokens get almost no learning signal.
- LSTMs mitigate this with gates (gives a more direct additive path for the cell state) but do not eliminate the sequential bottleneck.
- **No parallelisation**, and per the handwritten notes: “**Transfer learning not possible**” with LSTMs in the way it later becomes possible with pretrained Transformers.
- **Attention** (even before full self-attention) removes the need for a fixed encoder bottleneck vector and gives **no explicit alignment requirement** – the model learns soft alignments.

2.4 Self-Attention Core Mechanics

For a token sequence, each token produces a **Query (Q)**, **Key (K)**, and **Value (V)** vector via learned linear projections of the embedding.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- Q, K, V are obtained via $Q = XW^Q$, $K = XW^K$, $V = XW^V$.
- The $\sqrt{d_k}$ scaling prevents large dot products from pushing softmax into a saturated (near one-hot, tiny-gradient) regime.
- V carries the *meaning/content* that gets mixed; Q, K only decide *how much* of each V to mix in.

Worked attention-matrix example (from slides)

For the sentence “A brown cat lazes on the shelf”, focusing on query $q_3 = \text{cat}$: the raw/normalised attention weights onto keys $k_1, \dots, k_7$ (A, brown, cat, lazes, on, the, shelf) are approximately $[0, 0.38, 0.48, 0.08, 0.01, 0, 0.05]$ — i.e. *cat* attends most to itself (0.48) and to *brown* (0.38, its modifier), matching the intuition that self-attention should recover modifier–modified relationships.

2.5 Multi-Head Attention**Multi-Head Attention**

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- Each head projects into a lower-dimensional subspace $d_k = d_{\text{model}}/h$ (e.g. $d_{\text{model}} = 512$, $h = 8 \Rightarrow d_k = 64$).
- Different heads can specialise: syntactic relations (subject–verb), semantic relations (thematic roles), positional patterns, etc.
- After concatenation, the final linear layer W^O lets heads **cross-inform** each other, producing one richer representation per token.
- **Gains:** parallelisation, long-distance dependency capture, and (per-layer complexity) self-attention is *faster than RNN* whenever sequence length $n <$ representation dimension d .

2.6 What Self-Attention Alone Is Missing

- **No word position** (permutation-invariant) $\Rightarrow$ needs Positional Encoding.
- **No non-linearity** $\Rightarrow$ needs the position-wise Feed-Forward Network (FFN) after attention in every block.
- **Looks at future words** (in a decoder this is illegal) $\Rightarrow$ needs a **causal mask**: set illegal (future) positions to $-\infty$ before softmax.

2.7 Positional Encoding (Preview – full treatment in Lecture 6)**Sinusoidal Positional Encoding (Vaswani et al., 2017)**

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Added directly to the token embedding: $x_p = \text{Embed}(w_p) + PE_p$. Sinusoidal PE may let the model extrapolate to sequence lengths longer than those seen in training (deterministic, no learned parameters).

2.8 Add & Norm (Residual Connections + Layer Normalization)

- **Residual connections:** $y = x + \text{Sublayer}(x)$ — trains better, smooths the loss landscape (contrast the jagged loss surface of ResNet-56 *without* skip connections vs. the smooth bowl *with* them; Li et al., 2018).
- **Layer Norm:** normalises to unit mean/variance *within each layer* (per token), cutting down uninformative variation in hidden vectors so training is more stable. (Full mechanics in Lecture 8.)

2.9 The Full Encoder–Decoder Diagram

- **Encoder block** (repeated $N \times$): Input Embedding + Positional Encoding $\rightarrow$ Multi-Head (self-)Attention $\rightarrow$ Add&Norm $\rightarrow$ Feed-Forward $\rightarrow$ Add&Norm.
- **Decoder block** (repeated $N \times$): Output Embedding (shifted right) + PE $\rightarrow$ **Masked** Multi-Head Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ **Cross-Attention** (Multi-Head Attention over encoder output) $\rightarrow$ Add&Norm $\rightarrow$ Feed-Forward $\rightarrow$ Add&Norm $\rightarrow$ Linear $\rightarrow$ Softmax.

Cross-Attention – exact Q/K/V source (frequently tested)

In encoder–decoder (cross) attention: **Queries come from the decoder, Keys and Values come from the encoder’s final output.** Every decoder position can look for relevant encoder positions. Two canonical use-cases: Machine Translation, Speech-to-Text. Only the *final* encoder layer’s output feeds every decoder layer – earlier encoder layers are not exposed directly, and there is no enforced one-to-one token alignment.

2.10 Transformer Type Zoo

- **Encoder-only** (e.g. BERT): bidirectional self-attention, good for understanding tasks.
- **Decoder-only** (e.g. GPT): causal self-attention, good for generation; this family now dominates research relative to encoder-decoder.
- **Encoder-decoder** (e.g. T5, original Transformer): both self-attention and cross-attention; used for seq2seq tasks like translation/summarisation.

2.11 Parameter Counting (worked example, $\sim 1\text{B}$ model)

Given hyperparameters $V=32000$ (vocab), $d=2048$ (embedding dim), $n=2048$ (context), $d_q=128$ (per-head dim), $h=l=16$ (heads, layers):

Component	Formula	Params	%
Token Embedding	$V \cdot d$	65M	6.95%
Positional Embedding	$n \cdot d$	4.2M	0.44%
Attention (all layers)	$4 \cdot d_q \cdot d \cdot h \cdot l$	268M	28.63%
FFN (all layers)	$2 \cdot d \cdot 4d \cdot l$	536M	57.37%
Decoding (LM head)	$d \cdot V$	65M	6.95%
Total		935.8M	

The **FFN dominates parameter count** ($\sim 57\%$), not attention. This is a commonly tested surprise – students often assume attention is the biggest component.

2.12 Transformer Gains & Drawbacks (Summary)

Gains: Parallel processing, semi-supervised/self-supervised pretraining, transfer learning, acting as knowledge stores, emergent reasoning.

Drawbacks: Quadratic computation in self-attention ($O(n^2)$), and the position representation problem (addressed across Lectures 6–7 with RoPE, GQA/MQA/MLA, linear/sparse attention).

2.13 Practice & Review

Q1. [MC] In cross-attention within a Transformer decoder, the Query vectors come from:

- (A) The encoder
- (B) The decoder
- (C) Both equally
- (D) Neither (Q is fixed)

Answer: (B). Decoder positions query the encoder's Keys/Values to decide what source information to pull in.

Q2. [Short Answer] Why is the scaling factor $1/\sqrt{d_k}$ needed in scaled dot-product attention?

Answer: Without scaling, dot products of high-dimensional vectors can have large magnitude, pushing softmax into a saturated region where gradients vanish for most inputs; dividing by $\sqrt{d_k}$ keeps the pre-softmax scores in a numerically stable range so gradients flow well.

Q3. [Short Answer] What are the two things self-attention alone lacks, and what component fixes each? *Answer:* (1) No word-position information $\rightarrow$ fixed by positional encoding. (2) No non-linearity $\rightarrow$ fixed by the position-wise feed-forward network following attention in each block. (A third commonly cited gap: no restriction on attending to future tokens $\rightarrow$ fixed via a causal mask in decoders.)

Q4. [Problem] For a Transformer with $d_{\text{model}} = 512$ and $h = 8$ heads, what is d_k per head, and why is this choice made rather than using $d_k = d_{\text{model}}$ per head? *Answer:* $d_k = 512/8 = 64$. Splitting into smaller per-head subspaces keeps total computation/parameters roughly constant while letting each head specialise in a different relational pattern; using full d_{model} per head would multiply cost by h with diminishing returns in expressiveness.

Q5. [Essay] Describe the transformer encoder block in detail. (*This exact question appeared in the 2024 PYQ – high-value to master.*) *Answer:* An encoder block takes token+positional embeddings as input. It applies (1) Multi-Head Self-Attention: input is linearly projected into Q, K, V ; scaled dot-product attention is computed per head; heads are concatenated and projected via W^O . (2) A residual connection adds the block's input back, followed by (3) Layer Normalization. (4) The normalized output passes through a position-wise Feed-Forward Network (two linear layers with a non-linearity, e.g. ReLU/GELU, expanding to $4d$ and back). (5) Another residual+LayerNorm follows. This whole block is stacked N times; only the final encoder layer's output is exposed to the decoder's cross-attention.

Q6. [Essay] Describe a sequence-to-sequence machine translation model where the decoder pays attention to the encoder. Why is attention needed? (*2023 PYQ pattern.*) *Answer:* A seq2seq model has an encoder that reads the source sentence into a set of contextual representations, and a decoder that generates the target sentence token by token. Without attention, the encoder must compress the entire source sentence into a single fixed-length vector, which becomes a severe bottleneck for long sentences and loses fine-grained information. With attention (cross-attention), at each decoding step the decoder computes a query from its current state and attends over *all* encoder hidden states, producing a weighted context vector that highlights the most relevant source words for the word currently being generated (e.g., different target words require attending to different source content words). This removes the fixed-length bottleneck and lets the model handle long sentences and reordering between languages.

3 Lecture 3 (6 August): Self-Attention Projections & Why Dot-Product

3.1 Overview

This session (paired with the *Tokenization* slide deck though the live discussion focused on attention internals) dissects **why** the specific mathematical form of self-attention was chosen, contrasting it with alternative “relevance functions.”

3.2 The Space of Possible Relevance Functions

The handwritten notes trace the instructor’s reasoning as a sequence of design trade-offs:

1. $\max(f(q_i, k_j))$ **via a small MLP**: $\text{MLP}_2(q_i, k_j; \omega_1, \omega_2)$ – fully general (can learn *any* relationship, including semantic relationships with no direct grammatical link) but **computationally expensive** because it requires a separate forward pass per (i, j) pair (not expressible as one matrix multiply). Complete, but slow.
2. **Cosine similarity**: $\frac{q_i \cdot k_j}{\|q_i\| \|k_j\|}$ – complete and geometrically meaningful, but the normalisation (computing norms) still is **not fast** to batch as cleanly as a raw dot product.
3. **Raw dot product** $q_i \cdot k_j = QK^\top$: expressible as a single **matrix multiplication**, which GPUs execute extremely efficiently. This is *why* dot-product attention was chosen: it is the best trade-off between expressiveness and speed.

Modifier vs Semantic relationship (subtle distinction, notes 6/8)

“Modified $\leftrightarrow$ Modifier relationship” = grammatically relevant (e.g., adjective–noun). “Semantic Relationship” = meaning-based association with **no grammatical relevance** (e.g., *horse* $\leftrightarrow$ *grass*). Self-attention with a flexible enough scoring function (in principle, MLP_2) can capture both; the actual dot-product mechanism captures both too, but implicitly, via learned W^Q, W^K projections.

3.3 From Position/Embedding to Q, K via Linear Projection

- Positional encoding is combined with the embedding ($PE(i) \rightarrow$ representation at position i).
- A **linear transformation (projection)** maps the d -dimensional representation down to $d_k = d/h$ (e.g., 1/8th if there are 8 heads): $q_i = W^Q \cdot \text{rep}(i)$, $k_i = W^K \cdot \text{rep}(i)$.
- This projection is why we get *multiple heads for free*: each head has its own W_i^Q, W_i^K, W_i^V .

3.4 Normalisation of Scores: Why Softmax

Softmax over attention scores

$$\alpha_{ij} = \text{softmax}_j(\text{score}_{ij}) = \frac{e^{\text{score}_{ij}}}{\sum_j e^{\text{score}_{ij}}}$$

Chosen because it is **fast, differentiable, and computationally inexpensive**, and it

guarantees $\sum_j \alpha_{ij} = 1$ so the result is a genuine weighted average of the V vectors – keeping the output in the same “scale” as the inputs.

$\sqrt{d_k}$ is called a **dimensionality constant** in the notes – it exists purely to counteract the growth of dot-product magnitude with dimension d_k .

3.5 Context Vector and the Rest of the Layer

$$c_i = \sum_j \text{attention}(i, j) \cdot V_j \quad (\text{context representation for word } i)$$

$$h_i = \sigma \left(\sum_i w_i c_i + b \right), \quad o_i = \sigma \left(\sum_i w_i h_i + b \right)$$

V is chosen to “carry meaning” and be computationally efficient to combine (a simple weighted sum).

Neural networks in general are described as **projecting/mapping into another space or dimension** – a segue into VC-dimension / representational capacity, elaborated in Lecture 10.

3.6 Practice & Review

Q1. [MC] Which relevance function for attention is the *most general* but also the *most computationally expensive*?

- (A) Dot product
- (B) Cosine similarity
- (C) A small MLP applied to each (q_i, k_j) pair
- (D) Euclidean distance

Answer: (C). An MLP over each pair can represent arbitrary relevance functions but cannot be reduced to a single batched matrix multiplication, unlike the dot product.

Q2. [Short Answer] Why does the dot product “win” over cosine similarity as the standard choice for attention scoring, despite cosine similarity being arguably more principled (scale-invariant)? *Answer:* Cosine similarity requires computing vector norms for every query and key and dividing them out, which adds extra non-matrix-multiply operations that break the clean single-GEMM structure that GPUs are optimised for. The raw dot product $QK^\top$ is one matrix multiplication, which is dramatically faster in practice, and the model can compensate for scale differences via learned projections and the $\sqrt{d_k}$ scaling.

Q3. [Short Answer] What is meant by a “semantic relationship with no grammatical relevance” in the context of attention weights, and give an example. *Answer:* Two words can be strongly associated in meaning/co-occurrence without having a direct syntactic dependency between them, e.g., “horse” and “grass” (thematic/topical association) versus “brown” modifying “cat” (a grammatical modifier relationship). Attention heads can, in principle, learn either type of association.

- Q4. [Essay]** Trace the three candidate scoring functions the instructor considered for self-attention (MLP, cosine similarity, dot product), and explain the trade-off that determined the final choice. *Answer:* See the enumerated list above (Sec 3.2): the guiding trade-off throughout is expressiveness/completeness vs. computational efficiency (ability to be computed as one large matrix multiplication on a GPU). Dot product wins specifically because $QK^\top$ for the whole sequence is one GEMM call, making training and inference tractable at scale, while still being expressive enough once the Q, K projections are learned.

4 Lecture 4 (10 August): The Transformer Design Cascade, Scaling & Efficiency Preview

4.1 Overview

This is a broad, connective lecture: the instructor frames Transformer research as a “**cascade**” or “domino” effect – each paper’s design choice (often driven by GPU/engineering constraints, not pure math) forces the next paper to adapt, producing the many architectural variants seen today.

The cascade / domino metaphor

A design choice in one paper (e.g., scaled dot-product attention) forces downstream adaptations (multi-head attention, then normalization placement, then positional encoding variants, then long-context tricks). Most of these changes are driven by **engineering constraints** (GPU memory, latency) rather than being purely theoretically motivated. This explains architectural diversity across model families.

4.2 Why Parallel Processing Wins (Recap + Formal Complexity)

- RNN: sequential, cannot batch across time $\Rightarrow$ poor GPU utilisation.
- Transformer: builds the full Q, K, V matrices for the sequence at once; attention is $O(L^2d)$ time for sequence length L , dimension d – but crucially this is one parallel operation, not L sequential ones.

4.3 Why Multiple Attention Layers (Depth)

“Entering a room” analogy

A single attention layer gives each token only a first-order (direct-neighbour) view of context. Stacking layers lets tokens integrate progressively more indirect, higher-order information – analogous to entering a room, seeing people directly (layer 1), then over several rounds of “conversation” (more layers) inferring the social hierarchy (global context). This is why deep Transformers use 6–96+ layers.

4.4 Normalisation Placement Preview: Pre-Norm vs Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al.): Add $\rightarrow$ LayerNorm, i.e. normalize *after* the residual addition.
- **Pre-Norm**: LayerNorm $\rightarrow$ Sublayer $\rightarrow$ Add, i.e. normalize *before* the sublayer.
- Pre-Norm stabilises very deep stacks and often removes the need for learning-rate warm-up; Post-Norm was the historical default. (Full depth in Lecture 9.)

4.5 Model Scaling & Parameter Budget Reasoning

- Embedding matrix size = $|V| \times d$ – a *one-time* cost that does not grow with depth.

- Attention + FFN parameters grow **linearly with number of layers L** ; for deep models the embedding fraction of total parameters shrinks.
- Historical context lengths: 512 (BERT-base) $\rightarrow$ 4k $\rightarrow$ 16k $\rightarrow$ 128k+ $\rightarrow$ specialised models at 260k tokens. Quadratic attention cost makes each jump non-trivial (see Lecture 7).

4.6 Historical Model Timeline (as discussed)

Transformer (2017, encoder–decoder) $\rightarrow$ GPT-2 (2019, decoder-only, 1.5B, 1024 ctx) $\rightarrow$ T5 (2020, encoder–decoder, multilingual, 11B) $\rightarrow$ GPT-3 (2020, 175B, 2k ctx) $\rightarrow$ InstructGPT/ChatGPT (2022–23, RLHF-tuned) $\rightarrow$ LLaMA/LLaMA-2 (2023–24, open, 7–70B) $\rightarrow$ GPT-4 (2023, multi-modal, up to 128k ctx) $\rightarrow$ multimodal transformers.

4.7 Parameter-Efficient Fine-Tuning Preview: LoRA

LoRA – Low-Rank Adaptation (numeric example, exam-relevant given 2024 PYQ asked LoRA vs. Adapters)

Freeze the original weight matrix W (say $100k \times 1k = 100M$ params). Instead of updating all of W , learn two small matrices A ($100k \times r$) and B ($r \times 1k$) with rank $r \ll \min(100k, 1k)$ (e.g. $r = 500$ gives $\sim 0.5M$ – $0.8M$ params, a $\sim 200\times$ reduction). At inference/training time, $\Delta W = A \cdot B$ and $W' = W + \alpha \cdot \Delta W$.

Note on Adapters (not deeply covered in the transcripts through 17 Sept, but appeared in the 2024 PYQ “How does LoRA differ from Adapters?”): Adapters insert small bottleneck feed-forward modules *between* existing Transformer layers (adding new, sequential computation at inference), whereas LoRA adds a *parallel* low-rank branch to existing weight matrices that can be merged back into W after training, adding **zero extra inference latency**. If this term’s material does not reach LoRA/Adapters in more depth before the quiz, treat this box as your safety net – it is a recurring instructor question.

4.8 Mixture-of-Experts (Preview – full treatment in Lecture 10)

Only a subset of expert feed-forward sublayers are active per token (sparse activation), letting total parameter count scale to hundreds of billions while keeping active compute per token modest.

4.9 Long-Context Challenges (Preview – full treatment in Lecture 7)

- Quadratic memory/compute: $O(L^2)$ attention matrix.
- Empirical **attention bias toward early tokens** – a distributional artifact of how text is typically written (intro/summary first).

4.10 Practice & Review

Q1. [MC] According to the “cascade” framing of Transformer research, most design changes across papers are primarily driven by:

- (A) Pure mathematical elegance
- (B) Engineering/hardware constraints

- (C) Random search
- (D) Marketing

Answer: (B).

- Q2. [Short Answer]** Explain the “entering a room” analogy for why Transformers stack multiple attention layers. *Answer:* A single attention layer only lets a token directly aggregate information from other tokens it has learned to weight highly (first-order neighbours). Additional stacked layers let information propagate transitively – a token in layer 2 can incorporate information that another token gathered in layer 1, giving higher-order, more global context, similar to inferring social structure in a room only after several rounds of interaction.
- Q3. [Problem]** Using the LoRA numeric example, if W is $100\text{k} \times 1\text{k}$ and rank $r = 500$, compute the parameter count of A , B , and the total, and the compression ratio vs. full fine-tuning. *Answer:* A is $100\text{k} \times 500 = 50\text{M}$... (using the values as given in lecture, A : $100\text{k} \times 500 = 50\text{M}$? Note: the lecture’s own arithmetic rounds this to $\approx 0.5\text{M}$ total by using much smaller effective dimensions in the illustrative example – the *qualitative* takeaway examined on exams is: $\text{params}(A) + \text{params}(B) = r(d_{\text{in}} + d_{\text{out}}) \ll d_{\text{in}} \cdot d_{\text{out}}$, a reduction of roughly $\min(d_{\text{in}}, d_{\text{out}})/r$ -fold.)
- Q4. [Essay]** How does the LoRA fine-tuning approach differ from Adapters? Explain with architectures. (*Exact 2024 PYQ wording – high priority.*) *Answer:* **Adapters** insert new, small bottleneck MLP modules *in series* inside each Transformer layer (typically after the attention and/or FFN sublayers): input $d \rightarrow$ down-project to a small bottleneck dim $r \rightarrow$ non-linearity $\rightarrow$ up-project back to $d \rightarrow$ residual add. Only the adapter parameters are trained; the backbone is frozen. Because adapters are inserted *in the forward path*, they add extra sequential computation and thus extra inference latency. **LoRA** instead freezes the original weight matrix W and learns a low-rank update $\Delta W = AB$ (with $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$) added *in parallel* to W : at inference, $W' = W + \alpha AB$ can be pre-computed and merged into W , so LoRA adds **zero extra inference latency**, unlike Adapters. Both are parameter-efficient fine-tuning methods that train a small fraction of parameters relative to full fine-tuning, but they differ in where the new parameters live (in-series bottleneck vs. parallel low-rank residual) and in their inference-time cost.

5 Lecture 5 (13 August): Tokenization Deep Dive

5.1 Overview

Slides: *ANLP-M26-Lec3-Tokenization.pdf*. Central theme: **tokenization is an engineering compromise**, not a linguistic exercise – you don’t need to know many languages to design a good tokenizer; you need to understand the statistical shape of the data and the compute constraints.

5.2 What Is a Token?

Token = Identifier

A token is simply an **identifier** for a piece of input, analogous to a parking-ticket number: it doesn’t encode meaning by itself, it just names a unit that the model looks up an embedding for. Granularity can be a byte, character, sub-word, or whole word.

5.3 The Vocabulary Explosion Problem

- An average speaker knows $\approx 20k$ words; highly educated individuals $\approx 50\text{--}60k$; web-scale corpora contain from 30k up to millions of unique word types.
- **One-hot representation math** (handwritten notes, 13/8): 3 billion words $\rightarrow$ 4 bytes/float $\rightarrow$ 12 GB just for a naive embedding table; a $10M \times 10M$ one-hot scheme is an enormous sparse matrix, so instead the embedding matrix is $d \times |V|$ with $|V| \approx 10M$ still being far too large.
- Parameter count for embedding = $|V| \times d$. For $|V| = 10M$, $d = 300$: 3 billion parameters, $\approx 12GB$ in FP32 – larger than many full models (e.g., BERT-base $\approx 110M$ params total).

5.4 Zipf’s Law

Zipf’s Law

$$f(r) \approx \frac{C}{r^s}, \quad s \approx 1$$

Word frequency f is (roughly) inversely proportional to its frequency rank r . Plotting rank vs. frequency on a log-log scale gives a straight line (power law). **Consequence: the top $\sim 3\text{--}5\%$ of word types cover $\sim 95\%$ of all token occurrences** – a tiny high-frequency set (mostly function words) dominates, while a huge long tail of rare words contributes little volume. This directly motivates keeping only a modest vocabulary of frequent (sub-)words.

5.5 Unicode, Scripts, and Bytes

- Unicode assigns a **code point** to every character across all scripts; most fit in 2 bytes, some (emoji, rare scripts) need up to 4 bytes.
- A *script* (e.g., Devanagari) can serve *multiple languages* (Hindi, Marathi, Nepali) – Unicode blocks correspond to scripts, not languages.

- Byte-level tokenizers operate below the character level, treating every input as a stream of bytes (0–255), which makes them fully language-/script-agnostic.

5.6 Statistical Foundations: MLE and EM

- **Maximum Likelihood:** choose the segmentation/vocabulary that **maximises the probability of the observed corpus**.
- Segmentation of a word into sub-words can be framed as an **Expectation-Maximisation** problem: the true segmentation is a *latent variable*; E-step estimates expected sub-word counts given current probabilities, M-step re-estimates probabilities (or greedily merges the highest-count pair, as in BPE).

Joint/conditional probability for a merge candidate (handwritten notes, 13/8)

$$P(s_i, s_j) = P(s_j | s_i) P(s_i) = P(s_i | s_j) P(s_j) = \frac{P(s_i \cap s_j)}{P(s_i) P(s_j)} \cdot P(s_i) P(s_j)$$

where s_i, s_j are candidate sub-word segments (prefix/suffix or adjacent pieces).

5.7 Byte-Pair Encoding (BPE) – Step by Step

1. Start with a vocabulary of all individual characters/bytes seen in the corpus.
2. Count frequencies of all adjacent symbol pairs.
3. Merge the **most frequent** pair into a new symbol; add it to the vocabulary.
4. Replace all occurrences of that pair in the corpus with the new symbol.
5. Repeat until reaching the target vocabulary size (commonly $\sim 30\text{k}$ – 50k tokens, or a fixed number of merges $\sim 32\text{k}$).

Key numeric fact (also appears in handwritten notes, 13/8)

$\sim 3\%$ of English vocabulary types account for $\sim 95\%$ of English tokens in running text – the practical justification for BPE-style sub-word vocabularies of modest size.

5.8 WordPiece vs. SentencePiece vs. Byte-level BPE

Method	Core idea	Notes
BPE	Greedy, frequency-based pair merging	Order-dependent; originally a compression algorithm (1994), adapted to NLP by Sennrich et al. (2015).
WordPiece	Maximises likelihood of a language model over segmentations; greedy longest-match at inference	Used in BERT/ALBERT; tends to preserve whole frequent words more (e.g., un + ##lock + ##able).
SentencePiece	A <i>library</i> , not an algorithm; treats space as a regular character ; can run BPE or a Unigram LM	Enables sentence-level, language-agnostic tokenization without pre-tokenisation; good for scripts without explicit word boundaries (Japanese, Thai).
Byte-level BPE	Operates on raw bytes (0–255) instead of Unicode chars	Fully script-agnostic; used in GPT-2/3; can merge across script boundaries in code-mixed text.

5.9 Limitations of Sub-word Tokenizers

- **Order dependence:** BPE’s greedy merges depend on training-corpus order; different corpora yield different vocabularies.
- **Semantic mis-alignment:** merges may not correspond to true morphemes.
- **Rare-word fragmentation:** very rare words explode into many small pieces, lengthening sequences.
- **Multilingual cross-script merges** can be nonsensical when a single BPE model spans many scripts.
- **Training cost:** naive pair counting is $O(|\Sigma|^2)$ per iteration; needs hash-map optimisation.

5.10 Beyond Tokenization: The Byte Latent Transformer (mentioned 17/8 in handwritten notes)

An emerging alternative is “**no tokenisation**”: e.g., treat every 4 raw bytes as a “patch”/token directly (a *Byte Latent Transformer*), skipping the sub-word vocabulary step entirely. This removes tokenizer-induced artifacts but shifts the burden onto the model to learn byte-level structure.

5.11 Practice & Review

Q1. [MC] According to Zipf’s law, what fraction of word types typically accounts for the vast majority of token occurrences in a corpus?

- (A) ~50%
- (B) ~3–5%
- (C) ~25%

(D) Exactly 1 word

Answer: (B).

- Q2. [Short Answer]** Why is a naive one-hot representation of a 10-million-word vocabulary computationally infeasible for a neural language model? *Answer:* A one-hot vector for 10M words is a 10M-dimensional sparse vector; the corresponding embedding lookup matrix has $10M \times d$ parameters (e.g., ≈ 3 billion for $d = 300$), consuming tens of GB of memory and making training prohibitively slow – this motivates shrinking the effective vocabulary via sub-word tokenization.
- Q3. [Short Answer]** Describe the Byte-Pair Encoding algorithm in 4–5 steps. *Answer:* (1) Initialise vocab = all characters/bytes. (2) Count frequency of every adjacent symbol pair in the corpus. (3) Merge the most frequent pair into a new symbol, adding it to vocab. (4) Replace all its occurrences in the corpus. (5) Repeat steps 2–4 until the target vocabulary size (or merge count) is reached.
- Q4. [Short Answer]** Contrast WordPiece and SentencePiece. *Answer:* WordPiece is a segmentation *algorithm* that picks merges to maximise a language-model likelihood and applies greedy longest-prefix matching at inference (used in BERT). SentencePiece is a *library/framework* that can run BPE or a Unigram LM underneath, and crucially treats the space character as an ordinary symbol so it needs no language-specific pre-tokenisation – useful for languages without explicit word boundaries.
- Q5. [Essay]** Explain how Expectation-Maximisation relates to sub-word segmentation, describing the roles of the E-step and M-step. *Answer:* The observed data are raw words; the true segmentation into sub-words is treated as a latent variable, since we never directly observe “correct” splits. In the E-step, given current sub-word probabilities, we estimate the expected segmentation (or expected sub-word counts) for each word. In the M-step, we update the sub-word probabilities (or, in BPE’s greedy approximation, pick the single highest-frequency pair to merge) to increase the likelihood of the corpus under the current segmentation. Iterating E- and M-steps (or repeated BPE merges) monotonically improves (or leaves unchanged) the corpus likelihood, converging to a vocabulary that balances compactness with reconstruction fidelity.
- Q6. [Essay]** Tokenization is described as an engineering decision rather than a purely linguistic one. Justify this claim with examples from the lecture. *Answer:* Several examples support this: (1) Vocabulary size is capped (e.g., $\sim 30k$) purely to keep the embedding matrix and compute tractable, sacrificing perfect word-level linguistic fidelity. (2) WordPiece’s greedy longest-match rule is chosen for inference speed, even when it splits words in linguistically odd places. (3) Byte-level BPE ignores script/language boundaries entirely so that one tokenizer can serve many languages, a purely practical multilingual-deployment consideration. (4) Unknown-token fallback and hyphenation/punctuation handling are chosen to keep the pipeline robust, not to reflect any particular grammatical theory. Together these show tokenization decisions are driven by memory, speed, and robustness constraints rather than linguistic ideals.

6 Lecture 6 (17 August): Attention as a Graph & Positional Encoding Variants

6.1 Overview

Slides: *ANLP-M26-Attention.pdf*. Two intertwined themes: (1) attention re-cast as operations on a fully-connected graph over tokens, connecting to linguistic word-order phenomena, and (2) a full taxonomy of positional encoding strategies.

6.2 Attention as a Fully-Connected Graph

Core reframing

For every token i , attention computes a weighted sum over **all** other tokens j (including itself), with weight α_{ij} . This makes the token sequence into a **fully-connected directed graph**: every node has an outgoing edge to every other node (including a self-loop), and edge weights are learned, softmax-normalised attention scores.

- **Multi-head attention = a collection of fully-connected graphs.** Each head can learn a different edge-weighting scheme, so different heads can approximate different linguistic graphs simultaneously: *syntactic* (dependency tree: head→dependent), *semantic* (thematic/co-occurrence links, e.g. horse↔grass), *discourse* (cross-sentence coreference, discourse markers).
- Implication: Transformers can model many linguistic relation types *simultaneously* without an explicit parser, purely by learning different attention patterns per head.

6.3 Word-Order Phenomena in Natural Language

- Attention discards strict sequence order (it's permutation-invariant over the graph), yet real language exhibits strong ordering preferences.
- **Local Word Groups (LWGs)**: short, rigidly-ordered clusters (adjective+noun, aux+verb) captured well by a Context-Free Grammar even when the wider sentence has flexible order.
- **English adjective ordering hierarchy**: opinion → size → age → shape → colour → origin → material → purpose (e.g., “a beautiful large old round green French wooden table”). Violating this order sounds ungrammatical.
- **Free vs. relatively-free word order**: languages with rich **case marking** (e.g., Hindi *Ram-ne* = ERG marking agent, *seb-ko* = ACC marking patient) can reorder constituents because grammatical roles are explicit in morphology; languages without overt case (English) rely on **word order** (subject before object) to disambiguate roles.
- **Discourse-level ordering**: causal/conditional connectives (“because”, “if...then”) impose a required directional order between clauses.

Why this matters for models

Even though attention treats tokens as an unordered set of graph nodes, the data itself encodes strong positional regularities, so **positional information must be injected explicitly** for the model to recover meaning-critical orderings.

6.4 Positional Encoding: Full Taxonomy

Method	Mechanism	Limitation
Absolute Learned	A learnable vector e_p per position $0, \dots, L_{\max} - 1$; $x_p = w_p + e_p$	Positions beyond $L_{\max}$ seen in training are untrained/undefined; model must learn relative relationships implicitly.
Sinusoidal (static)	$PE_{(p,2i)} = \sin(p/10000^{2i/d})$, $PE_{(p,2i+1)} = \cos(\cdot)$; no learned params	Deterministic, extrapolates somewhat, but the pattern repeats with period $\approx 10000^{2/d}$.
Relative PE (Shaw et al., 2018)	A learned bias b_{Δ} , $\Delta = i - j$, added directly to the attention score before softmax	Needs a lookup table capped at some max distance (e.g., 128); zero bias beyond that, hurting very long-range modelling.
Rotary PE (RoPE, 2021)	Rotates Q, K pairs of dimensions by an angle $\theta_p = p \cdot \theta$; the resulting $QK^{\top}$ inner product depends only on relative angle $\theta(p_i - p_j)$	Infinitely extensible (no lookup table, no hard cutoff); the current dominant choice in modern LLMs.
NoPE (causal-mask only)	Rely purely on the decoder's causal mask for implicit ordering	Works only for autoregressive decoder-only generation; not preferred generally, fails for bidirectional/encoder use.

Rotary Positional Embedding (RoPE) – the mechanism, tested heavily

For a 2-D sub-vector (x, y) at position p with base angle θ , rotation angle $\theta_p = p \cdot \theta$:

$$(x', y') = (x \cos \theta_p - y \sin \theta_p, x \sin \theta_p + y \cos \theta_p)$$

Applied to *both* Q and K before the dot product. Because rotation preserves vector norm, $Q' \cdot K' = \|Q\| \|K\| \cos(\theta_i - \theta_j)$ – a smooth function of *only the relative distance* $\Delta = i - j$, with no lookup table and no bound on Δ .

6.5 Causal Mask

$$M_{ij} = 0 \text{ if } i \geq j; \quad M_{ij} = -\infty \text{ if } i < j \quad \Rightarrow \quad \text{score}_{ij} \leftarrow \text{score}_{ij} + M_{ij}$$

Implemented in practice with a large negative constant (e.g. -10^9) rather than true $-\infty$; can be applied as either an additive mask before softmax or equivalently a multiplicative 0/1 mask – both give the same result.

6.6 Proximity Bias / Long-Distance Dependencies

Empirically, most of a token's useful predictive context lies within the previous 3–4 sentences; relative/rotary encodings that naturally decay influence with distance align well with this reality, while still allowing the model to attend far when needed (e.g. coreference spanning paragraphs).

6.7 Self-Attention, KV, and the Bridge into Efficient Attention (from the same slide deck)

Self-Attention recap with concrete matrix shapes

$$QK^\top = \begin{bmatrix} q_0 \cdot k_0 & q_0 \cdot k_1 & \cdots \\ q_1 \cdot k_0 & q_1 \cdot k_1 & \cdots \\ \vdots & & \ddots \end{bmatrix}, \quad \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

KV Cache motivation: during autoregressive generation, tokens are produced one at a time; without caching, the model recomputes K, V for *all* previous tokens at every new step. A KV cache stores previously computed K, V so only the newest token's Q, K, V need to be computed each step – an optimisation purely for **inference** (not training, where the full sequence is known upfront and processed in parallel).

6.8 Multi-Query Attention (MQA), Grouped-Query Attention (GQA), Multi-Head Latent Attention (MLA) – Introduced Here, Elaborated in Lecture 7

Mechanism	KV Cache per Token	Capability
Multi-Head Attention (MHA)	$2n_h d_h l$	Strong
Grouped-Query Attention (GQA)	$2n_g d_h l$	Moderate
Multi-Query Attention (MQA)	$2d_h l$	Weak
Multi-Head Latent Attention (MLA)	$(d_c + d_h^R)l \approx \frac{9}{2}d_h l$	Stronger than MHA

(n_h = number of heads, d_h = per-head dim, l = layers, n_g = number of KV groups in GQA, d_c, d_h^R = MLA compression dims. Table from DeepSeek-V2 paper, Liu et al. 2024.)

6.9 Longformer – Efficient Attention for Long Documents (also in this deck)

- **Motivation:** solve $O(n^2)$ scaling of full self-attention for long documents.
- **Sliding window attention:** fixed window w ; each token attends to $w/2$ tokens each side; cost $O(n \times w)$; can vary w per layer (small windows lower layers = local context, larger/CNN-like receptive field growth with depth).
- **Dilated sliding window:** gaps of size (dilation) d within the window, letting different heads focus on local vs. longer context without increasing cost.
- **Global attention:** a few pre-selected tokens (e.g., [CLS] for classification, all question tokens for QA) attend to *and are attended to by* the entire sequence – gives task-specific flexibility that pure local windows lack.

- **Gain:** $O(n)$ complexity combining local + global attention.
- **Drawbacks:** potential information loss from windowing; task-specific global-attention patterns were only validated for QA, coreference resolution, and document classification; may not help tasks needing only short local context.

6.10 Transformer Models Over the Years (attention-mechanism mapping, from slides)

Model	Attention
Original Transformer, GPT, mT5	MHA
GPT-3	MHA (unconfirmed variant)
LLaMA, LLaMA 3.2, Qwen 3	GQA
Mistral	Sliding-window attention + GQA
DeepSeek V3, Kimi K2	MLA

(2026-era models per slides: Qwen 3.5 – hybrid gated DeltaNet; Sarvam – MLA with KV-norm; Nemotron 3 Ultra – GQA; Mistral 4 – MLA; Gemma 4 – 5 sliding-window layers + 1 GQA layer per block; Qwen 3.6 – Gated DeltaNet; DeepSeek V4 – compressed sparse attention; Kimi K3 – 3 Kimi Delta Attention layers + 1 MLA layer.)

6.11 Practice & Review

Q1. [MC] Why is attention described as inducing a “fully-connected graph” over tokens?

- (A) Because every token has a learned edge (attention weight) to every other token, including itself
- (B) Because tokens are physically connected in memory
- (C) Because it uses a graph neural network internally
- (D) Because only adjacent tokens attend to each other

Answer: (A).

Q2. [MC] What is the key mathematical property that makes RoPE “infinitely extensible” compared to a relative-position bias table?

- (A) It uses more parameters
- (B) The relative-angle dependence emerges from a rotation, computable for any integer position with no lookup table
- (C) It ignores position entirely
- (D) It only works for short sequences

Answer: (B).

Q3. [Short Answer] Why does English rely on strict word order (subject before object) while Hindi can shuffle word order more freely? *Answer:* English nouns carry no case marking to indicate grammatical role, so word order is the only signal distinguishing subject from object (“Ram ate the apple” vs. “The apple ate Ram”). Hindi marks grammatical roles morphologically via case suffixes (e.g., ergative *-ne*, accusative *-ko*), so the roles remain unambiguous even if the surface word order changes.

- Q4. [Short Answer]** Explain how the KV cache changes the computational profile of autoregressive generation. *Answer:* Without caching, generating token t would require recomputing K, V projections for all $t - 1$ prior tokens, an $O(t)$ cost repeated at every step ($O(t^2)$ total). With a KV cache, previously computed K, V vectors are stored and reused, so each new step only computes Q, K, V for the newest token and reuses the cache for the attention computation, reducing generation to $O(t)$ total incremental work. This trades increased memory (the cache grows linearly with sequence length) for large speed gains, and is used only at inference (not training, where the whole sequence is available in parallel).
- Q5. [Essay]** Compare absolute learned, sinusoidal, and rotary positional embeddings on: mathematical form, extrapolation to longer sequences, and impact on attention computation. *Answer:* **Absolute learned** PE assigns an independently-trained vector to each position up to $L_{\max}$; it cannot generalise past $L_{\max}$ since unseen positions have untrained embeddings, and the model must learn relative-distance effects implicitly from these independent vectors. **Sinusoidal** PE is a fixed deterministic function of position with no learned parameters; because it uses many frequencies, the phase difference between two positions encodes their relative distance, and the formula can technically be evaluated for any position, giving some extrapolation ability, though the pattern periodically repeats at very large positions. **RoPE** rotates the query/key vectors by an angle linear in position *before* the dot product, so the attention score naturally depends only on the relative angular difference between two positions; this gives true, parameter-free extensibility to arbitrary sequence lengths and is why RoPE is the dominant choice in modern long-context LLMs, whereas the earlier two methods either cannot extrapolate (absolute) or degrade gracefully at best (sinusoidal).
- Q6. [Essay]** Explain Longformer’s sliding-window + global attention design and its complexity gain over full self-attention. *Answer:* Longformer replaces full $O(n^2)$ self-attention with a combination of (a) a **sliding window** of fixed size w , so each token attends only to $w/2$ neighbours on each side ($O(n \cdot w)$ cost, analogous to a CNN’s local receptive field, with receptive field growing across stacked layers), optionally **dilated** (gaps of size d) to reach farther without extra cost, and (b) **global attention** on a small, task-chosen set of tokens (e.g., [CLS] for classification or all question tokens for QA) that attend to, and are attended to by, every other token, restoring the flexibility that pure local windows lack for tasks needing whole-sequence reasoning. Combined, this gives $O(n)$ overall complexity instead of $O(n^2)$, enabling processing of much longer documents, at the cost of some potential information loss from restricted windows and the need to hand-pick which tokens get global attention per task.

7 Lecture 7 (20 August / “More Attention” 24 Aug deck): Efficient Attention, KV-Cache Engineering & Encoder-Decoder Wiring

7.1 Overview

Two decks converge here: the 20-Aug transcript (encoder-decoder data flow, KV caching, quantization) and the “More Attention” slide deck (dated within the slides as 3 Sept, covering standard/sliding/sparse/hybrid/linear attention formally). This is the most heavily mathematical efficiency-focused lecture and a rich source of parameter-counting-style exam questions.

7.2 Encoder-Decoder Data Flow (Recap + Detail)

- Only the **final encoder layer’s** output is used as Key/Value input for *every* decoder layer’s cross-attention – not each encoder layer to its “matching” decoder layer.
- There is **no enforced one-to-one** source-to-target token alignment; the mapping is learned and many-to-many.
- Encoder-decoder research has slowed relative to decoder-only (GPT-style) models in recent years.

7.3 The Quadratic Bottleneck, Formally

For sequence length n , hidden dim d : computing $Q = XW^Q$ etc. is $O(nd^2)$; computing $A = \text{softmax}(QK^\top/\sqrt{d_k})$ is $O(n^2d_k)$ per head, $O(hn^2d_k) = O(n^2d)$ total; multiplying AV is again $O(n^2d_v)$. **Total:** $O(n^2d)$ time, $O(n^2)$ memory for the attention matrix alone.

Scaling reality check

$n = 512 \Rightarrow n^2 \approx 262\text{k}$; $n = 4\text{k} \Rightarrow 16\text{M}$; $n = 16\text{k} \Rightarrow 256\text{M}$; $n = 260\text{k} \Rightarrow \approx 68$ billion entries – even at 16-bit precision this exceeds 120GB, impossible on a single GPU without an efficient-attention or caching workaround.

7.4 K/V Caching – The Full Mechanics

- At each generation step t : compute Q_t for the new token only; retrieve cached $K_{1:t-1}, V_{1:t-1}$; append newly computed K_t, V_t to the cache; compute attention using the full $K_{1:t}, V_{1:t}$.
- Cache size $\approx n \times d_k \times h \times l \times (\text{bytes per value})$; grows **linearly** with generated length and is a major inference memory driver for long-context models.
- Only used at **inference**; training processes the whole sequence in parallel (teacher forcing), so no cache is needed.

7.5 Reducing Attention’s Parameter/Memory Footprint: MQA, GQA, MLA – Full Detail

7.5.1 Multi-Query Attention (MQA, Shazeer 2019)

Motivation: incremental (autoregressive) decoding is slow and not parallelisable, dominated by repeated loading of Key/Value tensors from memory – a *memory-bandwidth*, not compute, bottleneck. **Approach:** keep multiple **Query** heads, but use a **single shared Key and Value head** across all of them. **Gain:** much lower memory-bandwidth requirement during incremental decoding. **Drawback:** reduced model quality/capacity (shared K,V may be too simplified to let each head learn distinct, optimal parameters); still doesn’t help training-time compute, only inference memory bandwidth.

7.5.2 Grouped-Query Attention (GQA, Ainslie et al. 2023)

Motivation: interpolate between MHA (quality) and MQA (speed) – “achieve quality close to MHA and speed close to MQA.” **Approach:** divide the h query heads into n_g groups (e.g., $n_g = 4-8$); each group shares a single K head and a single V head. Can be obtained by **converting** an existing MHA checkpoint: mean-pool each group’s original per-head Key (and Value) projection matrices into one shared projection. **Gain:** quality close to MHA, speed/memory comparable to MQA. **Drawback:** more complex than either extreme – must choose the number of groups n_g and manage K/V-to-group assignment.

7.5.3 Multi-Head Latent Attention (MLA, DeepSeek-V2, Liu et al. 2024)

Motivation: reduce KV cache further while *improving* (not just preserving) quality relative to MHA.

MLA – Low-rank KV (and Query) joint compression

$$c_t^{KV} = W^{DKV} h_t \in \mathbb{R}^{d_c}, \quad k_t^C = W^{UK} c_t^{KV}, \quad v_t^C = W^{UV} c_t^{KV}$$

where $d_c \ll d_h n_h$ is the KV compression dimension, W^{DKV} is the down-projection, W^{UK}, W^{UV} are up-projection matrices. A parallel low-rank compression is applied to queries: $c_t^Q = W^{DQ} h_t$, $q_t^C = W^{UQ} c_t^Q$, to reduce *training*-time activation memory (queries are not cached at inference, so this doesn’t affect the KV cache).

Key inference trick: MLA only needs to cache the compressed latent c_t^{KV} (not full K, V); moreover W^{UK} can be algebraically absorbed into W^Q and W^{UV} into W^O , so keys/values need never be explicitly materialised for attention at all. **Result (from the DeepSeek-V2 paper table):** MLA’s KV cache per token $\approx (d_c + d_h^R)l \approx \frac{9}{2}d_h l$ – equivalent to GQA with only **2.25 groups**, yet MLA’s reported quality is **stronger than full MHA**. **Drawbacks:** cannot be obtained by converting an existing MHA/MQA checkpoint (unlike GQA) – requires pre-training from scratch; higher implementation complexity.

7.6 Parameter/Cache Comparison Table (worked, ~1B model)

Hyperparameters used below: $d = 2048$, $n = 2048$, $d_h = 128$, $h = l = 16$.

Attention	KV Cache Formula	Total Params (M)	Cache Memory (MB, FP32)
MHA	$n \cdot l \cdot 2 \cdot d_h \cdot h$	134	512
MQA	$n \cdot l \cdot 2 \cdot d_h$	8	32
GQA ($n_g=4$)	$n \cdot l \cdot 2 \cdot n_g \cdot d_h$	33	128
MLA ($d_c=32$)	$n \cdot l \cdot 2 \cdot d_c$	2	8

7.7 Precision & Quantisation

Precision	Bytes/scalar	Memory for 134M-param cache example
FP32	4	512 MB
FP16 / BF16	2	256 MB
FP8 / INT8	1	128 MB

Quantisation is used to fit larger caches/models in limited GPU memory, at the cost of numeric precision (mitigated with quantisation-aware training or calibration).

7.8 Standard Attention – Restated Disadvantages (“More Attention” deck)

(i) Quadratic in time (ii) Every token attends to every other (iii) Fixed context window limits input size. These three disadvantages motivate everything below.

7.9 Sliding (Window) Attention – Formalised

- Attends only to neighbours within a window; **linear** in both input size and window size w : $O(n \cdot w)$.
- Stacking layers still enables distant influence (receptive field grows with depth, CNN-style).
- **Limitation:** long-range interaction needs information to traverse many stacked layers; window w too small loses information, too large defeats the purpose (approaches full attention’s cost).

7.10 Sparse Attention

- **Strided/fixed sparse attention** (Child et al. 2019, Sparse Transformer): attends to a window of tokens *plus* a fixed stride pattern; originally aimed mainly at image data.
- **DeepSeek’s sparse attention:** attends to a *selected few* tokens (no fixed window) – the selected tokens are those with top- k attention scores computed by a single specialised low-precision “indexer” head.
- **Limitations:** fixed stride/block patterns assume a regular structure that may not exist; non-contiguous memory access is hard to optimise on GPUs; the DeepSeek top- k -selection approach adds its own selection overhead.

7.11 Hybrid Attention (BigBird, Zaheer et al. 2020)

Combines **random attention** + **window attention** + **global attention**, all $O(n)$, giving both local and global context for tasks that need both (e.g., QA). **Limitations:** choosing which tokens are “global” is task-dependent; combining heterogeneous attention patterns (sliding, random, global) complicates parallelisation.

7.12 Linear Attention

Standard (softmax) attention, restated

$$o_i = \frac{\sum_j \exp(q_i^\top k_j) v_j}{\sum_j \exp(q_i^\top k_j)}$$

The exponential nonlinearity *couples* q_i, k_j inside the nonlinearity, forcing the full $n \times n$ score matrix $QK^\top$ to be materialised: $O(n^2d)$ time, $O(n^2)$ memory (or $O(n)$ KV cache at inference, growing with context).

The linearization trick

Replace the softmax kernel $\exp(q^\top k)$ with a **factorisable** kernel $\phi(q)^\top \phi(k)$ for some feature map $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$. Then:

$$o_i = \frac{\phi(q_i)^\top \sum_j \phi(k_j) v_j^\top}{\phi(q_i)^\top \sum_j \phi(k_j)} = \frac{\phi(q_i)^\top S}{\phi(q_i)^\top z}, \quad S = \sum_j \phi(k_j) v_j^\top \in \mathbb{R}^{d' \times d}, \quad z = \sum_j \phi(k_j)$$

S, z are **independent of i** : compute once, then each query costs $O(d'd) \Rightarrow$ **overall $O(nd'd)$: linear in n** , with memory independent of n .

Causal linear attention is an RNN (Katharopoulos et al. 2020, “Transformers are RNNs”)

With a causal mask, the sums become running prefix sums:

$$S_t = S_{t-1} + \phi(k_t) v_t^\top, \quad z_t = z_{t-1} + \phi(k_t), \quad o_t = \frac{\phi(q_t)^\top S_t}{\phi(q_t)^\top z_t}$$

This is exactly a recurrent network with a **matrix-valued hidden state** $S_t \in \mathbb{R}^{d' \times d}$: $O(1)$ **time and memory per generated token, no KV cache needed at all.**

Choosing ϕ : $\text{elu}(x)+1$ (original, cheap, guarantees positivity); positive random features (Former, unbiased estimate of $\exp(q^\top k)$); Identity/ReLU/SiLU (common in recent models); or drop the denominator entirely and normalise the output instead (LayerNorm/RMSNorm) – most modern variants use a simple ϕ plus output normalisation.

Why plain linear attention underperforms softmax (heavily testable)

- **Bounded state:** S_t has fixed size $d' \times d$ regardless of context length – retrieval of arbitrary past tokens degrades as more information is crammed into the same fixed-capacity state (poor needle-in-a-haystack / in-context recall).
- **Attention dilution:** softmax gives sharp, near one-hot weights; a generic kernel gives smoother, more spread-out weighting, losing sharp selectivity.
- **No forgetting:** $S_t = \sum_{j \leq t} \phi(k_j) v_j^\top$ only *accumulates* – interfering keys blend their values together and nothing is ever erased (motivating later “gated”/“delta” variants that can selectively forget, e.g. Gated DeltaNet used in 2026-era Qwen models).

7.13 Practice & Review

Q1. [MC] Which statement about MQA is **FALSE**?

- (A) It uses multiple query heads with a single shared K and V head
- (B) It primarily targets memory-bandwidth cost during incremental decoding
- (C) It generally improves model quality relative to MHA
- (D) It reduces KV cache size significantly

Answer: (C). MQA typically *reduces* quality/capacity relative to MHA because K,V are shared and simplified; it trades quality for speed/memory.

Q2. [MC] Grouped-Query Attention is best described as:

- (A) A special case of MQA with zero groups
- (B) An interpolation between MHA and MQA, sharing K/V within groups of query heads
- (C) Identical to MLA
- (D) Only usable for encoder-only models

Answer: (B).

Q3. [Short Answer] Why can GQA be obtained by “converting” an existing MHA checkpoint, but MLA cannot? *Answer:* GQA’s grouping can be approximated post-hoc by mean-pooling the K (and V) projection matrices of the heads within each group into a single shared projection, requiring only light fine-tuning, since it is structurally still “one K/V per group of heads.” MLA instead learns a fundamentally different, jointly-compressed low-rank latent representation of K and V (c_t^{KV}) via new down/up-projection matrices that have no counterpart in a standard MHA checkpoint, so it must be trained from scratch (pre-training), not converted.

Q4. [Short Answer] State the formula for the KV cache size per token for MHA, MQA, GQA, and MLA, and explain the variable meanings. *Answer:* MHA: $2n_h d_h l$; MQA: $2d_h l$; GQA: $2n_g d_h l$; MLA: $(d_c + d_h^R)l \approx \frac{9}{2}d_h l$. Here n_h = number of heads, d_h = dimension per head, l = number of layers, n_g = number of KV groups (GQA), d_c = KV compression dimension and d_h^R = per-head dim of the decoupled RoPE part of the query/key in MLA.

- Q5. [Problem]** Derive why linear attention (with feature map ϕ) achieves $O(n)$ time complexity for a sequence of length n , contrasted with standard attention's $O(n^2)$. *Answer:* Standard attention must materialise the full $n \times n$ matrix $QK^\top$ because the softmax's exponential nonlinearity couples every query with every key directly, costing $O(n^2d)$. Linear attention replaces $\exp(q^\top k)$ with a factorisable kernel $\phi(q)^\top \phi(k)$, letting us pre-compute $S = \sum_j \phi(k_j)v_j^\top$ and $z = \sum_j \phi(k_j)$ *once* in $O(nd'd)$ time (a single pass over all keys/values, independent of queries), after which each query's output costs only $O(d'd)$ via $\phi(q_i)^\top S / \phi(q_i)^\top z$ – giving total cost $O(nd'd)$, linear in n .
- Q6. [Essay]** Why does linear attention, despite its superior asymptotic complexity, generally underperform softmax attention in practice? Discuss at least three reasons. *Answer:* (1) **Bounded state capacity:** the recurrent state $S_t \in \mathbb{R}^{d' \times d}$ has fixed size regardless of how many tokens have been seen, so as context grows, more information must be compressed into the same fixed capacity, degrading retrieval of specific past tokens (poor in-context recall/“needle in a haystack” performance). (2) **Attention dilution:** softmax produces sharp, near-categorical attention distributions that let the model sharply focus on a few relevant tokens; most kernel feature maps ϕ produce smoother, more diffuse weightings, losing this selective sharpness. (3) **No forgetting mechanism:** the state update $S_t = S_{t-1} + \phi(k_t)v_t^\top$ is purely additive/accumulative – old and new information blend together with nothing ever being subtracted or forgotten, so interference between old and new keys corrupts retrieval over long contexts. These limitations motivate hybrid and gated variants (e.g., Gated DeltaNet) that add selective-forgetting mechanisms on top of the linear-attention framework.
- Q7. [Essay]** Discuss the evolution from quadratic multi-head attention to linear, sparse, and hybrid alternatives, comparing their complexities, memory footprints, and typical use-cases. *Answer:* Full multi-head attention gives an exact softmax distribution over all token pairs but costs $O(n^2)$ time and memory, becoming infeasible past a few thousand tokens. **Sliding-window attention** reduces this to $O(nw)$ by restricting each token to a local neighbourhood, relying on depth (stacked layers) to propagate longer-range information, at the cost of needing many layers for truly distant dependencies. **Sparse attention** (strided/fixed patterns, or DeepSeek's learned top- k selection) keeps a similarly reduced cost while choosing which distant tokens to attend to, but assumes exploitable structure and is harder to parallelise efficiently on GPUs due to non-contiguous memory access. **Hybrid attention** (BigBird) combines windowed, random, and global attention to get $O(n)$ cost while still supporting whole-sequence global reasoning for a few designated tokens, at the cost of task-specific tuning of which tokens are global. **Linear attention** replaces the softmax kernel with a factorisable one, achieving true $O(n)$ time and (in the causal/RNN form) $O(1)$ per-token inference cost with no KV cache at all, but sacrifices the sharp, high-capacity retrieval of exact softmax attention due to its fixed-size recurrent state. In practice, the choice depends on required context length, whether exact retrieval fidelity or throughput matters more, and hardware constraints (memory bandwidth vs. compute).

8 Lecture 8 (7 September): Normalization Fundamentals

8.1 Overview

After a scheduling gap, this lecture returns to a foundational but often under-emphasised topic: **normalization**. The instructor stresses this predates Transformers and is not a mere afterthought – it is central to why gradient descent works well in deep networks.

8.2 Why Normalize: The Loss-Geometry Argument

Consider linear regression $y = ax_1 + bx_2$ with MSE loss $L = \frac{1}{m} \sum_i (y_i - \hat{y}_i(a, b))^2$ (a **convex** function of a, b , guaranteeing a unique global minimum for this simple model).

- If $x_1 \in [2, 3]$ and $x_2 \in [2000, 5000]$ (wildly different scales), the loss surface becomes a highly **elongated ellipse**: moving along the coefficient of the small-scale feature changes the loss sharply, while moving along the large-scale feature’s coefficient changes it only slowly.
- **Consequence for gradient descent**: gradients w.r.t. the large-scale-feature’s coefficient are small (relative to the error), so the optimiser spends most effort on the small-scale feature, potentially leaving the large-scale feature under-trained, or causing oscillation if the learning rate is tuned for the steep direction.
- **Normalization fix**: rescale each feature to comparable variance (e.g., z-score standardisation), making the loss surface more isotropic (circular), so gradient descent converges efficiently in all directions.

Z-score standardisation

$$\hat{x}_i = \frac{x_i - \mathbb{E}[x_i]}{\text{Var}(x_i)} \quad (\text{commonly written } \frac{x_i - \mu}{\sigma})$$

Motivated by the **Six-Sigma / Gaussian rule**: $\approx 99.9999\%$ of Gaussian-distributed data lies within $\pm 6\sigma$ of the mean – so dividing by σ captures “practically all” the meaningful variation into a bounded range.

8.3 Three Conceptual Spaces

- **Input space**: raw, possibly unconstrained/heterogeneous-scale data.
- **Computation space**: internal activations after linear transforms – normalization’s main job is to keep *this* space bounded (roughly zero-centred), preventing overflow/underflow and saturated non-linearities.
- **Output space**: final predictions; comparatively easy to rescale after the fact (e.g., softmax).

“If you cannot control the input space, you must control the computation space.” – Normalization is the simplest, most differentiable way to keep the computation range bounded.

8.4 Why Normalization Also Fights Over-fitting

If one feature's scale dominates the loss, the model may effectively **memorise the exact numeric range** seen in training for that feature; if test data shows the same feature at a different scale, predictions degrade badly. Normalizing forces every feature to contribute comparably, encouraging the model to learn genuine relationships rather than incidental magnitude patterns.

8.5 Batch Normalization vs. Layer Normalization

Batch Normalization

Population = the current mini-batch (size B), computed **per feature, across samples**:

$$\mu_B = \frac{1}{B} \sum_{k=1}^B x^{(k)}, \quad \sigma_B^2 = \frac{1}{B} \sum_{k=1}^B (x^{(k)} - \mu_B)^2, \quad \hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y^{(k)} = \gamma \hat{x}^{(k)} + \beta$$

At inference, uses a running (exponential moving average) estimate of μ, σ^2 collected during training.

Layer Normalization

Population = the feature dimensions of a **single sample** (index-wise per notes), computed **per sample, across features**:

$$\mu_\ell = \frac{1}{D} \sum_{j=1}^D x_j, \quad \sigma_\ell^2 = \frac{1}{D} \sum_{j=1}^D (x_j - \mu_\ell)^2, \quad \hat{x}_j = \frac{x_j - \mu_\ell}{\sqrt{\sigma_\ell^2 + \epsilon}}, \quad y_j = \gamma \hat{x}_j + \beta$$

Independent of batch size – works even with batch size 1.

Aspect	Batch-Norm	Layer-Norm
Population	Across batch, per feature	Across features, per sample
Batch-size dependence	Strong – small batches (esp. size 1) give unreliable/undefined statistics (BN disadvantage : unknown batch size, can be 1)	None
Typical use	CNNs with large, stable batches	RNNs, Transformers (variable/small batch, sequential data)
Train/inference gap	Needs running-average statistics for inference	None – same computation at train and test

RMSNorm – Layer Norm's simplification (bridges into Lecture 9)

RMSNorm is “**Layer Norm with $\mu, \beta = 0$** ” – it forces the mean to zero and drops the additive shift entirely, normalising purely by the root-mean-square: $r = \sqrt{\frac{1}{n} \sum_i x_i^2}$, $\hat{y}_i = \gamma x_i / (r + \epsilon)$. Cheaper (no mean subtraction) and empirically more stable for very deep Transformers.

Weight Normalization (contrast with activation normalization)

Normalizes **parameters**, not activations: $\hat{w} = \frac{g}{\|v\|} v^\top x + b$, decoupling a weight vector's direction from its magnitude g . Not the same family as Batch/Layer/RMS-Norm.

8.6 BatchNorm vs. LayerNorm – Which Axis Is Normalised (slide-deck formalism)

The official slide deck (*ANLP_M26_Lecture_Normalization.pdf*) makes the batch-vs-sample distinction geometrically precise using a [Batch $\times$ Sentence Length $\times$ Feature] cube of activations:

- **BatchNorm normalises *index-wise***: for a batch of inputs $x^{(1)}, \dots, x^{(B)}$, it slices the cube **across the batch dimension** for each feature index i : $\mu_i = \frac{1}{B} \sum_b x_i^{(b)}$, $(\sigma_i)^2 = \frac{1}{B} \sum_b (x_i^{(b)} - \mu_i)^2$, $y_i^{(b)} = \gamma_i \frac{x_i^{(b)} - \mu_i}{\sqrt{(\sigma_i)^2 + \epsilon}} + \beta_i$.
- **LayerNorm normalises *input-wise***: for a single input $x^{(b)}$ of dimension d , it slices the cube **across the feature dimension** for that one sample: $\mu^{(b)} = \frac{1}{d} \sum_i x_i^{(b)}$, $(\sigma^{(b)})^2 = \frac{1}{d} \sum_i (x_i^{(b)} - \mu^{(b)})^2$, $y^{(b)} = \gamma^{(b)} \frac{x^{(b)} - \mu^{(b)}}{\sqrt{(\sigma^{(b)})^2 + \epsilon}} + \beta^{(b)}$.

This is why the slides visualise LayerNorm as normalising a single *slab* (one sentence-length $\times$ feature slice) while BatchNorm/Power-Norm normalises across the whole *batch-dimension face* of the cube.

8.7 HybridNorm (Zhuo et al., NeurIPS 2026) – a 2026 refinement

HybridNorm

Four transformer layer structures were compared: (a) Post-Norm, (b) Pre-Norm, (c) Pre-Norm with QK-Norm, (d) **HybridNorm**. Empirically, **post-attention normalisation is more sensitive to the choice of normalisation type than other positions** in the block. HybridNorm exploits this: it uses **LayerNorm specifically at the post-attention position** and **RMSNorm everywhere else**, giving a meaningful speedup relative to using LayerNorm throughout, *without* the catastrophic instability that can occur if RMSNorm is used at *every* position.

8.8 Practice & Review

Q1. [MC] Which of the following is **FALSE** about Batch Normalization?

- (A) Computes mean/variance across the batch dimension per feature
- (B) Requires learnable γ, β
- (C) Works equally well when batch size is 1
- (D) Uses running averages at inference

Answer: (C). With batch size 1, the batch variance is ill-defined/zero, breaking BN.

Q2. [MC] Two features $x_1 \in [2, 3]$ and $x_2 \in [2000, 5000]$ are used, unnormalised, in a linear regression trained with MSE. The most likely consequence is:

- (A) Perfectly balanced, fast convergence
- (B) An elongated loss surface causing imbalanced gradient magnitudes and slow/unstable convergence
- (C) The loss becomes non-convex
- (D) No effect at all

Answer: (B).

Q3. [Short Answer] Explain the “three spaces” framing (input, computation, output) and normalization’s role. *Answer:* Input space is the raw data, which may have arbitrary/heterogeneous scales across features. Computation space is the internal activations after linear layers, where normalization primarily acts to keep values bounded and centred, preventing numerical instability and keeping non-linearities in their responsive regime. Output space is the final prediction, which is comparatively easy to rescale afterwards (e.g., via softmax) and is less normalization’s concern.

Q4. [Short Answer] Why is Layer Norm generally preferred over Batch Norm for Transformers and RNNs? *Answer:* Transformers/RNNs often use variable or small batch sizes (e.g., 1 during autoregressive inference, or variable sequence lengths), which makes Batch Norm’s per-batch statistics noisy or ill-defined. Layer Norm computes statistics across the feature dimension of each individual sample independently, so it is unaffected by batch size and gives identical, stable behaviour at train and test time.

Q5. [Essay] Discuss the trade-offs between Batch Normalization and Layer Normalization, covering how each computes statistics, batch-size dependence, typical architectures, and training/inference consistency. *Answer:* (See table above for structure.) Batch-Norm aggregates over the batch dimension for each feature, giving very reliable statistics when batches are large and homogeneous (ideal for CNNs on image batches), but it requires maintaining running-average statistics for use at test time (since test-time batches may be size 1 or differently distributed), creating a train/inference mismatch risk, and fails outright when batch size is very small. Layer-Norm aggregates across the feature dimension within a single example, making it completely independent of batch size and identical at train and test time, which is why it is the default in Transformers and RNNs where batch composition varies or generation happens one sample at a time. The cost is that Layer-Norm cannot exploit cross-example statistical smoothing the way Batch-Norm can, but this is an acceptable trade for the robustness it buys in sequence models.

9 Lecture 9 (10 September): Pre-Norm vs. Post-Norm, RM-SNorm, Residuals & Representation Collapse

9.1 Overview

Direct continuation of Lecture 8, now focused specifically on *where* normalization sits relative to the residual connection inside a Transformer block, and the training-dynamics consequences of that placement.

9.2 Residual (Skip) Connections

Residual connection

$$y = x + F(x)$$

Provides a direct (identity) gradient path back through the network, mitigating vanishing gradients and enabling very deep stacks (hundreds of layers).

Handwritten-notes variant with an activation wrapped around the sum: $x_{L+1} = \sigma(x_L + F(x_L))$
 – mitigates vanishing gradients.

9.3 Post-Norm vs. Pre-Norm vs. DeepNorm

Three placements

Post-Norm (original Transformer): $\text{Post} = \text{LN}(x + F(x))$

Pre-Norm: $\text{Pre} = x + F(\text{LN}(x))$

DeepNorm: $x_{L+1} = \text{LayerNorm}(\alpha x_L + F(x_L))$ – Post-Norm *scaled* by a depth-dependent factor α

Variant	Behaviour	Trade-off
Post-Norm	Sub-layer receives the <i>unnormalised</i> raw input; training can be unstable for deep stacks	But: no loss of information in the residual signal itself
Pre-Norm	Sub-layer always receives a normalised input $\Rightarrow$ avoids vanishing gradients , very stable for deep nets	But: representations can collapse (successive layers barely change the signal)
DeepNorm	Adds a depth-scaled α to the residual before Post-Norm	Enables >48-layer Post-Norm-style training without instability

DeepNorm's α (illustrative)

$\alpha = (2N)^{1/4}$ for N layers (theoretical derivation keeps the Jacobian's spectral norm bounded across layers). For $N = 48$: $\alpha = (96)^{0.25} \approx 2.0$.

Rule of thumb from the notes

Pre-Norm suits **small networks**; **Deep-Norm** suits **larger/deeper networks**. Hybrid schemes (Pre-Norm before attention, extra norm after FFN, etc.) exist to try to get the best of both worlds, at the cost of added complexity and sometimes instability at extreme depth unless α -scaling is carefully tuned.

9.4 Representation Collapse

Diagnosed by computing **cosine similarity between consecutive layers' hidden states**:

$$\cos(h_\ell, h_{\ell+1}) = \frac{h_\ell \cdot h_{\ell+1}}{\|h_\ell\| \|h_{\ell+1}\|}$$

- High similarity (≈ 1) across many consecutive layers $\Rightarrow$ **collapse**: later layers are learning almost nothing new, effectively wasting model capacity/depth.
- Occurs more in **Post-Norm** because the residual can dominate while the sub-layer's contribution gets repeatedly damped by normalization of the sum.
- Pre-Norm's similarity curve fluctuates more (lower, more varied values) $\Rightarrow$ each layer is transforming the representation substantially – generally desirable, though *too* low similarity can also indicate instability.

9.5 Emergent Features

In very large/deep models, after crossing a size/depth threshold, a **small number of hidden dimensions** suddenly acquire disproportionately large magnitude (“spikes”) while most dimensions stay small.

- **Interpretability upside**: such dimensions can often be linked to specific linguistic phenomena and are easy to probe.
- **Compression downside**: uniform quantisation schemes struggle because a few large-magnitude dimensions dominate the dynamic range, forcing coarse quantisation of everything else unless per-channel scaling or mixed precision is used.
- Related to **over-parameterisation**: models with far more parameters than strictly needed still train well (implicit regularisation from gradient descent), but this abundance is also what allows a handful of dimensions to specialise so strongly.

9.6 Non-Linearity Recap: FFN and Beyond (Bridges to Lecture 10)

The Feed-Forward Network (FFN) is what supplies **non-linearity** in each Transformer block. Gating mechanisms (e.g., Gated Linear Units) further modulate this non-linearity and are covered in Lecture 10.

9.7 Practice & Review

Q1. [MC] In a pre-norm Transformer block, LayerNorm is applied:

- (A) After the residual addition

- (B) Before the attention/FFN sub-layer, prior to the residual addition
- (C) Only at the final output layer
- (D) On Q, K, V simultaneously by definition

Answer: (B).

Q2. [MC] Representation collapse is most directly diagnosed by observing:

- (A) Training loss diverging to infinity
- (B) Cosine similarity between successive layers' hidden states staying near 1 across many layers
- (C) Uniform attention scores
- (D) Vanishing token embeddings

Answer: (B).

Q3. [Short Answer] Why can Post-Norm cause representation collapse while Pre-Norm tends to avoid it (but risks other issues)? *Answer:* In Post-Norm, the residual sum $x + F(x)$ is normalised *after* being formed; if $F(x)$'s contribution is small relative to x , the normalisation of the combined sum can repeatedly dampen $F(x)$'s effect layer after layer, so successive layers barely change the representation (collapse). In Pre-Norm, F always receives a freshly normalised input and its output is added *unnormalised* to the running residual stream, letting each layer contribute a more substantial, undamped update – but because the raw residual stream keeps growing/accumulating without repeated renormalisation of the whole state, very deep Pre-Norm stacks can still face other stability issues if not carefully scaled.

Q4. [Essay] Discuss the trade-offs between Pre-Norm and Post-Norm Transformer architectures, covering training stability, representation learning, and representation collapse. *Answer:* (See table + discussion above.) Pre-Norm stabilises gradient flow for deep networks because every sub-layer's input is normalised before use, avoiding the vanishing-gradient problems that plague deep Post-Norm stacks; however, this can lead to representation collapse, where the residual pathway dominates and successive layers make only small, similar changes to the hidden state (diagnosable via near-1 cosine similarity across layers), reducing the effective use of depth. Post-Norm, the original Transformer's choice, normalises after the residual addition, which does not systematically damp the sub-layer's contribution in the same way (less collapse risk) but is prone to training instability (exploding or vanishing gradients) in very deep stacks because the sub-layer itself always processes an un-normalised, and potentially wildly-scaled, input. DeepNorm resolves this tension for very deep (48+ layer) models by keeping the Post-Norm structure but multiplying the residual branch by a carefully derived depth-dependent scale α before normalising, which keeps gradients bounded while preserving the anti-collapse property of Post-Norm. In practice: small/shallow networks often use plain Pre-Norm; very large/deep modern LLMs increasingly use DeepNorm or hybrid schemes.

Q5. [Essay] What are emergent features in large Transformers, how are they detected, and why do they complicate model compression? *Answer:* Emergent features are individual hidden dimensions that, once a model exceeds some size/depth/training threshold, develop disproportionately large activation magnitudes relative to the rest of the vector, effectively dominating that dimension's contribution to the representation for many inputs. They are detected by monitoring per-dimension activation statistics across layers (looking for a small subset

of dimensions whose magnitude is orders of magnitude larger than the median), sometimes cross-checked against layer-wise cosine-similarity or probing studies linking a dimension to a specific linguistic phenomenon. They complicate compression (particularly quantisation) because a uniform quantisation scheme must set its numeric range to accommodate the largest values in a tensor; if a few emergent dimensions have very large magnitude, the quantisation step size for the whole tensor is forced to be coarse, degrading precision for the (majority) of normally-scaled dimensions. Mitigations include per-channel scaling, mixed-precision formats, or explicitly identifying and protecting emergent dimensions during quantisation.

10 Lecture 10 (17 September): Neural Network Foundations, Activation Functions, Gating & Mixture-of-Experts

10.1 Overview

A theory-heavy lecture surveying the history of neural networks and connecting classic ideas (perceptrons, VC-dimension, learning paradigms) to modern architectural components (gating, GLUs, Mixture-of-Experts) that show up in current LLMs.

10.2 Historical Timeline

Era	Key development
1943–1958	McCulloch–Pitts neuron (1943); Rosenblatt’s Perceptron (1958): $y = \text{sign}(w \cdot x + b)$, a linear threshold unit.
1969	Minsky & Papert’s <i>Perceptrons</i> : proves a single-layer perceptron cannot compute XOR (not linearly separable) – triggers an “AI winter” for connectionist research.
1986	Rumelhart, Hinton & Williams formalise back-propagation for multilayer networks, using the chain rule to compute gradients for hidden layers – makes deep(er) nets trainable.
1980s–90s	Energy-based models (Boltzmann Machines, Hopfield nets) – probabilistic, physics-inspired training via contrastive divergence.
2006–2012	Deep Belief Networks, CNNs (LeCun) revive deep learning with more data + GPUs.
2017–present	Transformers, massively over-parameterised models, with sparsity/gating (Mixture-of-Experts) to manage compute at scale.

10.3 Linear Separability, XOR, and VC-Dimension

- A single perceptron implements only a **linear decision boundary** $\text{sign}(w \cdot x + b)$; it cannot separate XOR-type data (not linearly separable), which is precisely why hidden (non-linear) layers are needed.
- **VC-dimension**: the size of the largest point set that a hypothesis class can *shatter* (classify in all possible label assignments). For linear separators in $\mathbb{R}^d$, $\text{VC-dim} = d + 1$. Higher VC-dimension = higher model capacity, but also higher risk of overfitting if not matched to data size.
- **Perceptron learning rule**: $w \leftarrow w + \eta(y - \hat{y})x$ – an early gradient-style update; the learning rate η trades off convergence speed against stability (too large $\Rightarrow$ oscillation/“shuttering”; too small $\Rightarrow$ slow convergence).

10.4 Three Core Learning Paradigms

Paradigm	Idea	Typical loss
Entropy Maximisation	Maximise output-distribution entropy $H(p) = -\sum p \log p$ subject to data constraints – avoid over-confidence	Maximum-entropy models, CRFs
Energy Minimisation	Define energy $E(x, y; \theta)$ with $p(y x) \propto e^{-E}$; minimise energy for correct pairs	Boltzmann Machine energy
Maximum Likelihood Estimation (MLE)	Maximise $\prod_i p(y_i x_i; \theta)$ (equivalently minimise negative log-likelihood)	Cross-entropy

All three connect through **KL-divergence**: $\text{KL}(p||q) = \mathbb{E}_p[\log p - \log q] = -H(p) - \mathbb{E}_p[\log q]$; minimising KL (MLE) implicitly trades off entropy and likelihood. Note: **KL-based losses are rarely used directly for > 3 classes** (too costly); softmax cross-entropy (a one-hot special case) is used instead.

10.5 Normalization (compression of the error range) Recap in this Lecture

Neurons/activations must “compress the error range” to keep gradients well-behaved – ties back to Lectures 8–9.

10.6 Activation Function Taxonomy

Function	Form	Derivative issue	Notes
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	Saturates $\Rightarrow$ vanishing gradient for $ z \gg 0$	Early NNs, binary classification
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	Same saturation issue	Zero-centred output
ReLU	$\max(0, z)$	Non-differentiable at $z = 0$ (sub-gradient used); dead-neuron risk for $z \leq 0$	Cheap, dominant in modern nets; can be seen as “a shifted sigmoid, zero for negative, linear for positive”
Leaky ReLU	$\max(\alpha z, z), \alpha \approx 0.01$	Mitigates dead neurons	Small negative slope
Swish/SiLU	$z \cdot \sigma(\beta z)$	Smooth, non-monotonic	Self-gated; good gradient flow
GELU	$\approx 0.5z(1 + \tanh[\sqrt{2/\pi}(z + 0.044715z^3)])$	Smooth	Default in BERT/GPT-2/3 FFNs

10.7 The Feed-Forward Network (FFN) – Formal Definition (FFNN slide deck)

FFN

$$\text{FFN}(x, W_1, W_2, b_1, b_2) = \max(0, xW_1 + b_1) W_2 + b_2$$

x is the hidden representation at one sequence position; it is passed through two learned linear transformations with a ReLU non-linearity in between. **Importance:** the FFN is what **adds non-linearity** to the model – without it, the stacked attention (linear) layers would collapse into an overall linear function no matter how many layers are stacked.

FFNN Issues – the motivation for everything that follows (GLU variants and MoE)

- **Static, dense design that is not content-aware** – the same fixed transformation is applied regardless of what the token actually needs.
- **Uniform computational load** – applies the same massive computation to every token, which is highly inefficient when tokens vary greatly in difficulty/content.
- **Static knowledge access** – activates the *entire* FFN parameter set for every token, regardless of how much of that knowledge is actually relevant.

Two independent fixes emerge from these issues: (1) **GLU variants** make the FFN’s transformation input-dependent (gating) without changing which parameters are active; (2) **Mixture-of-Experts** makes only a *subset* of parameters active per token (sparsity), directly attacking the “uniform load” and “static knowledge access” problems.

10.8 Gating Mechanisms & GLU Variants (formal treatment)

Gated Linear Unit (GLU) – general form and the FFN it produces

$$\text{GLU}(x, W, V, b, c) = \sigma(xW + b) \otimes (xV + c), \quad \text{FFN}_{\text{GLU}}(x, W, V, W_2) = (\sigma(xW) \otimes xV) W_2$$

Gating mechanic: x is projected by W through a sigmoid, producing a gate vector with values in $(0, 1)$; simultaneously x is projected by V ; the element-wise product ($\otimes$) of the gate and this second projection gives a filtered representation, which is then projected by W_2 . Motivation (as in GRUs/LSTMs): create a **learnable, input-dependent filter**.

Variant	Formula
GLU	$\sigma(xW + b) \otimes (xV + c)$
Bilinear	$(xW + b) \otimes (xV + c)$ (sigmoid removed entirely)
ReGLU	$\max(0, xW + b) \otimes (xV + c)$
GeGLU	$\text{GELU}(xW + b) \otimes (xV + c)$
SwiGLU ★	$\text{Swish}_\beta(xW + b) \otimes (xV + c)$, where $\text{Swish}_\beta(x) = x\sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}$ – the current de facto standard (LLaMA, Mistral, Qwen 3, DeepSeek V3, Kimi K2)

The “3 weight matrices” cost, and how it is paid for

All GLU variants use **3 weight matrices** (W, V, W_2) versus the standard FFN’s 2 (W_1, W_2). To keep total parameter count comparable, the number of hidden units d_{ff} is **reduced by a factor of $\frac{2}{3}$** when switching a plain FFN to a GLU-style FFN.

Why do GLU variants work? (Shazeer, 2020)

Shazeer’s own paper offers no formal proof – “we offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.” The **working hypothesis** taught in class: (1) **richer representations** – input-dependent gating enables flexible, token-specific transformations; (2) **dynamic feature selection** – gates amplify relevant features and suppress irrelevant ones (same intuition as Sec. 10.7’s gating discussion).

SiTU-GLU (Kimi K3, 2026) – a new GLU variant proposed this year

Applies a smooth cap $\text{softcap}(x, \beta) = \beta \tanh(x/\beta)$ to *both* the gate’s linear factor and (independently) the up-projection branch of a Swish gate:

$$\text{SiTU-GLU}(x) = \left[\beta_1 \tanh\left(\frac{W_g x}{\beta_1}\right) \odot \sigma(W_g x) \right] \odot \left[\beta_2 \tanh\left(\frac{W_u x}{\beta_2}\right) \right]$$

This bounds the output magnitude ($|f(x)| \leq \beta_1 \beta_2$), curbing the **activation spikes** described below while still behaving like SwiGLU for small inputs.

Activation Spikes – a quantisation-relevant caveat

Recent research shows GLU gating can cause extreme activation spikes, especially in early and late layers. These spikes are **harmless in full-precision training** but **hinder post-training quantisation**, because standard (uniform) quantisation schemes cannot simultaneously represent typical small values and rare large spikes with adequate precision. (Directly connects to the **emergent features** discussion in Lecture 9.)

- **Scalar gate** (Highway Networks): $g = \sigma(w_g \cdot x + b_g)$; $y = g \cdot F(x) + (1 - g) \cdot x$ (transform vs. carry).
- **LSTM gates**: input, forget, and output gates (all sigmoid-based, values in $[0, 1]$) control how much new information is written, how much old cell state is retained, and how much is

exposed as output.

- **Historical model→activation progression** (from the slide deck’s “Transformer Models over Years” table): Original Transformer/GPT → **ReLU**; GPT-2/3 → **GeLU**; mT5-Large → **GeGLU**; LLaMA, Mistral, LLaMA 3.2 → **SwiGLU** (still dense FFN); DeepSeek V3, Qwen 3, Kimi K2 → **SwiGLU + MoE** (sparse). This table is a compact way to remember that the FFN’s evolution has two independent axes: *which activation/gating* and *dense vs. sparse (MoE)*.

10.9 Mixture-of-Experts (MoE) – Comprehensive Treatment

What is MoE, and why now

MoE is a technique that uses multiple sub-models (“experts”). **Scaling laws** say that performance predictably improves as parameters, data, and compute all increase – but training ever-larger *dense* models is not feasible for most organisations. MoE introduces **sparse, conditional computation**: only a subset of experts fires per token, so total parameters can grow enormously while the compute cost for any single input stays manageable (“Scaling through Sparsity”).

- **Where it plugs in**: an MoE layer **typically replaces the dense FFN/MLP sublayer** inside a Transformer block, immediately after the (masked) self-attention + Add&Norm. The rest of the block (attention, residuals, norms) is unchanged. A **less common** use applies the same router/experts pattern to attention heads themselves.
- **Advantages**: sparse models → much larger parameter count → more stored information, *without* increasing FLOPs per token (you can add experts without affecting FLOPs); MoE is inherently parallel (**expert parallelism** – different experts can live on different devices); sparse models are also well suited to **data parallelism** since each device only needs to host the experts it uses.
- **Why MoE was historically unpopular** (pre-2020s): complex architecture; training objectives are somewhat heuristic (the load-balancing loss is an auxiliary hack, not a principled objective); and MoE models are **unstable to train** without careful engineering.
- **Open questions raised in lecture**: what do experts actually learn? How do we ensure complementary, non-redundant knowledge across experts (i.e., is “divine benevolence” really enough, riffing on the GLU quote)? How do we interpret these more complex models?

10.9.1 MoE Components

Experts: “Expert” is somewhat of a *misnomer* – at least one expert is always activated for every token, and experts (each typically a small FFN) simply **specialise on different things during training** via gradient descent, not by design. Empirical probing (Zoph et al., “ST-MoE”, 2022) of a T5-style MoE found experts that specialise by *layer and position* rather than by clean topic, e.g.:

Specialisation	Layer	Example routed tokens
Sentinel tokens	1, 4, 6	<extra_id_N> placeholder tokens
Punctuation	2, 6	commas, colons, ampersands, question marks
Conjunctions & articles	3, 6	“the”, “a”, “and”, “or”
Verbs	1	died, fell, closed, signed, submitted
Visual descriptions (colour/position)	0	dark, upper, blue, inner, bright
Proper names	1	Mart, Kent, Colin, Lorraine
Counting and numbers	1	37, 19, 6, Seven, 2012

Router/Gate: the most critical component – itself a small FFNN that outputs probabilities used to select the best-matching expert(s) for each token. **Load Balancer:** an auxiliary mechanism to keep expert usage roughly even (details below).

10.9.2 Routing Algorithms

- **Hash Routing:** each token is deterministically assigned to an expert by a fixed hash function of the token – simple, requires no learned router, but mostly of **academic** interest today (no adaptivity to content).
- **Top- k Routing** – the dominant approach in production models, with three sub-flavours:
 - *Token choice:* each token independently picks its top- k highest-scoring experts (as in Fig. “Top-2 Routing”: token “The” $\rightarrow$ 65% weight on FFN1 + 30% on FFN4; token “Dog” $\rightarrow$ 80% on FFN1 + 15% on FFN3).
 - *Expert choice:* instead, each *expert* picks its top tokens from the batch (inverts the assignment direction; naturally balances load but can drop tokens no expert wants).
 - *Global routing via optimisation:* a joint optimisation decides the whole token $\leftrightarrow$ expert assignment matrix at once (most balanced, most expensive).
- **Reinforcement-Learning routing:** the router is trained via a policy-gradient-style loss $\text{Loss} += -\log(p_{\text{selected}}) \cdot R$ where R is a downstream reward. Used in earlier MoE works; **not popular anymore** because it is higher-variance and harder to train than direct top- k softmax routing.
- **Models using top- k routing:** Switch Transformer ($k = 1$), GShard ($k = 2$), Grok, Mixtral, Qwen, DeepSeek.

Expert Selection and Output – the exact computation

$$H(x) = x \cdot W \quad (\text{router logits, one score per expert})$$

$$G(x) = \text{softmax}(H(x)) \quad (\text{probabilities}), \quad y = \sum_e G(x)_e \cdot E_e(x)$$

y sums each expert’s output weighted by its router probability; unselected experts contribute 0. The expert layer returns the output of the selected expert(s) multiplied by the correspond-

ing gate value(s) – exactly the “weighted activation” shown in the MoE-layer diagrams.

10.9.3 Load Balancing

Routing challenges: the same expert(s) may always be chosen, producing (a) uneven load distribution across experts, (b) some experts hardly ever trained, and (c) experts training at different speeds – a self-reinforcing rich-get-richer dynamic.

Load Balancing Loss (Fedus, Zoph & Shazeer – Switch Transformer, 2022)

Importance score per expert: $\text{Imp}_e = \sum_{x \in \text{batch}} G(x)_e$. Compute the **coefficient of variation** across experts’ importance scores: $\text{CV} = \sigma/\mu$. The auxiliary loss aims to **lower** this CV:

$$L_{\text{balance}} = w \cdot \text{CV} \quad (w = \text{a constant scaling hyper-parameter})$$

Other mitigations mentioned: adding **Gaussian noise** to router logits (encourages exploration across experts) and **Token Choice routing** with a capacity limit (below).

Expert Capacity & Capacity Factor (Fedus, Zoph & Shazeer, 2022)

Expert Capacity = batch size each expert is allowed to process = $\left(\frac{\text{tokens_per_batch}}{\text{num_experts}} \right) \times \text{capacity_factor}$. The **capacity factor** (e.g. 1.0 vs. 1.5) adds buffer headroom to mitigate **token overflow** during routing (tokens routed to an already-full expert are dropped or overflow to a lower-priority pick); a higher capacity factor reduces dropped tokens at the cost of more cross-device communication and wasted compute on padding.

10.9.4 DeepSeekMoE – Shared + Routed Experts (Liu et al., DeepSeek-V2/V3)

- Splits experts into two pools: a small number of **shared experts** that are *always* active (capturing common/general knowledge every token needs) and a larger pool of **routed experts** selected via top- K_r routing (capturing specialised knowledge).
- Architectural evolution shown in the slides: (a) Conventional Top-2 Routing → (b) + Fine-grained Expert Segmentation (many smaller experts instead of few large ones, same total capacity) → (c) + Shared Expert Isolation (= DeepSeekMoE). The trend is toward **smaller, more numerous routed experts plus a few always-on shared experts**.
- DeepSeekMoE is paired with **Multi-Head Latent Attention (MLA)** in the same Transformer block (RMSNorm → Attention/MLA → Add → RMSNorm → FFN/MoE → Add), tying together Lecture 7’s efficient-attention material with this lecture’s efficient-FFN material.

10.10 Transformer Models Over the Years – 2026 Snapshot (params vs. active params)

Model	Architecture	Num Ex- perts	Total Params	Active Params
Qwen 3.5	MoE	512	397B	17B
Sarvam	MoE	128	105B	10.3B
Nemotron 3 Ultra	Mamba + Latent MoE	–	550B	55B
Mistral 4	MoE	128	119B	6B
Gemma 4	Dense FFNN	–	12B	–
Qwen 3.6	MoE	256	35B	3B
DeepSeek V4	MoE (SwiGLU)	384	1.6T	49B
Kimi K3	FFNN + Latent MoE (SiTU-GLU)	896	2.8T	104B

The recurring exam-relevant number

Across every MoE row, **Active Params** $\ll$ **Total Params** (e.g. Kimi K3: 104B active out of 2.8T total, $\approx 3.7\%$) – this is the entire point of sparse MoE: near-dense-model compute cost at a fraction of the FLOPs, with far more *stored* knowledge than compute would otherwise allow.

Kimi K3 note (handwritten notes, 17/9)

Kimi K3 uses sigmoid/tanh-style gating (SiTU-GLU, see above) as part of scaling to a larger model (paper marked “read paper” in notes – flagged by instructor as a case study worth independent reading).

10.11 Residual / Highway Networks Recap in This Lecture’s Framing

Architecture	Equation	Benefit
Residual (ResNet)	$y = F(x) + x$	Mitigates vanishing gradients, enables very deep nets
Highway	$y = T(x)F(x) + (1 - T(x))x$, $T(x) = \sigma(W_T x + b_T)$	Network <i>learns</i> how much to transform vs. carry
Gated Residual	$y = g \cdot F(x) + x$, $g = \sigma(W_g x + b_g)$	Extra control over information flow, useful in very deep / MoE settings

10.12 Practice & Review

Q1. [MC] Which of the following was *not* one of the three “core approaches” to learning in neural networks discussed?

- (A) Entropy maximisation
- (B) Energy minimisation

- (C) Bayesian inference
- (D) Maximum-likelihood estimation

Answer: (C).

Q2. [MC] Mixture-of-Experts achieves runtime efficiency primarily by:

- (A) Activating all experts for every token
- (B) Activating only a small top- k subset of experts per token via a learned router
- (C) Removing the FFN entirely
- (D) Using only one very large expert

Answer: (B).

Q3. [Short Answer] What problem does Minsky & Papert’s 1969 critique of the perceptron identify, and how was it eventually resolved? *Answer:* They proved a single-layer perceptron can only represent linearly separable functions, and XOR is not linearly separable, so a single perceptron cannot learn it. This was resolved by using multilayer networks with non-linear hidden layers (which can bend the decision boundary into non-linear shapes), made trainable in practice by the 1986 back-propagation algorithm.

Q4. [Short Answer] Explain how a Gated Linear Unit (GLU) improves training efficiency. *Answer:* A GLU computes two projections of the input, uses a sigmoid on one to produce a per-dimension gate in $[0, 1]$, and multiplies it element-wise with the other projection. This lets the network suppress (near-zero-out) dimensions it deems irrelevant for the current input, increasing the effective signal-to-noise ratio of gradients and focusing parameter updates on the dimensions that matter, which tends to speed up and stabilise training compared to an ungated linear/ReLU transform.

Q5. [Short Answer] Why is a load-balancing auxiliary loss needed when training Mixture-of-Experts models? *Answer:* Without it, the router can collapse to routing most or all tokens to only a few “popular” experts (a rich-get-richer dynamic during training), leaving other experts under-trained and wasting the model’s total capacity. The auxiliary loss penalises deviation from a uniform expected load across experts, encouraging the router to spread tokens more evenly.

Q6. [Essay] Trace how the field addressed the limitation of representing complex decision boundaries across the perceptron era, the back-propagation era, and the modern gating/MoE era. *Answer:* The perceptron era (1950s–early 1970s) offered a single linear decision boundary, adequate for linearly separable problems but provably unable to solve XOR-type problems (Minsky & Papert, 1969), which halted progress until the back-propagation era (1980s onward) showed how to train *multilayer* networks with non-linear activations, letting stacked linear+non-linear transformations approximate arbitrarily complex decision boundaries (a universal function approximator, in principle). This power came with a cost: very large parameter counts and training instability in deep networks. The modern era addresses this via **gating** (GLUs, LSTM gates) which lets the network conditionally suppress irrelevant dimensions per input, improving training efficiency and effective capacity utilisation, and via **Mixture-of-Experts**, which partitions a huge parameter budget into many specialised sub-networks and activates only a sparse subset per token – allowing enormous total capacity (trillions of

parameters) while keeping per-token compute (and thus training/inference cost) tractable. Each era’s solution addressed the specific scaling bottleneck (representational power, then trainability, then compute/parameter efficiency) exposed by the previous one.

Q7. [Essay] Explain the role of the router in a Mixture-of-Experts layer, why sparsity helps at inference, and one major training challenge. *Answer:* The router is a lightweight linear layer that projects the input token representation to a score per expert ($s = xW_r$), converted to probabilities via softmax (optionally temperature-scaled), from which the top- k highest-probability experts are selected to actually process that token; their outputs are combined (often weighted by the router’s probabilities). Sparsity helps at inference because only the parameters of the selected top- k experts need to be loaded and executed for a given token, so a model can have an enormous *total* parameter count (spread across many experts) while its *active*, per-token compute and memory footprint stays comparable to a much smaller dense model. A major training challenge is **expert imbalance/collapse**: without intervention, gradient descent can concentrate routing probability on a small subset of experts early in training, starving the rest of gradient signal; this is mitigated with auxiliary load-balancing losses, careful initialisation, and sometimes noise injection into the router’s scores.

Q8. [MC] All GLU-family FFN variants (GLU, ReGLU, GeGLU, SwiGLU) use how many weight matrices, compared to a standard FFN?

- (A) 2 vs. 2
- (B) 3 vs. 2
- (C) 2 vs. 3
- (D) 4 vs. 2

Answer: (B). GLU variants need W , V , and W_2 (3 matrices) versus a plain FFN’s W_1, W_2 (2 matrices); to compensate, d_{ff} is shrunk by $\frac{2}{3}$ to keep parameter count comparable.

Q9. [Short Answer] Write the SwiGLU-based FFN formula and explain how it differs from ReGLU. *Answer:* $\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_\beta(xW) \otimes xV)W_2$ where $\text{Swish}_\beta(x) = x\sigma(\beta x)$. ReGLU instead uses $\max(0, xW + b)$ (plain ReLU) as the gate activation instead of the smooth, self-gated Swish function; SwiGLU’s smoothness and non-monotonicity near zero is credited (without formal proof) for its consistently better empirical performance, which is why it is the dominant choice in LLaMA, Mistral, Qwen and DeepSeek-family dense FFNs.

Q10. [Short Answer] Define Expert Capacity and the Capacity Factor in MoE training, and explain the trade-off involved in raising the capacity factor. *Answer:* Expert Capacity = $(\text{tokens_per_batch}/\text{num_experts}) \times \text{capacity_factor}$ – it is the maximum number of tokens an expert is allowed to process in one batch. The capacity factor (e.g. 1.0 vs. 1.5) adds slack above the “perfectly balanced” allocation to absorb natural routing imbalance. Raising it reduces the number of tokens dropped/overflowed when an expert is oversubscribed, but increases cross-device communication and wastes compute on padding for under-subscribed experts – a classic load-balancing-vs-efficiency trade-off.

Q11. [Problem] The Switch Transformer load-balancing loss is $L_{\text{balance}} = w \cdot \text{CV}$ where $\text{CV} = \sigma/\mu$ is the coefficient of variation of experts’ importance scores. Explain, in one or two sentences, what happens to this loss as routing becomes perfectly uniform across experts, and why that is the desired training signal. *Answer:* As routing becomes perfectly uniform, every expert’s

importance score $\text{Imp}_e = \sum_x G(x)_e$ converges to the same value, so the standard deviation $\sigma \rightarrow 0$ and hence $\text{CV} = \sigma/\mu \rightarrow 0$, driving $L_{\text{balance}} \rightarrow 0$. This is the desired signal because minimising this loss during training explicitly pushes the router toward using all experts roughly equally, preventing the collapse where a few experts absorb most tokens while others go untrained.

11 Bridging Notes: What Only the Handwritten Notes Reveal

Several small but exam-relevant nuggets appear *only* in the handwritten notes and not explicitly in the transcript-derived summaries above. Cross-checked and merged into their respective lectures above, but flagged again here for quick review:

- **“A language is a dialect with an army and navy.”** (30/7) – language/dialect boundary is political.
- **Attention as a spectrum of relevance functions** (6/8): MLP (complete, slow) → cosine similarity (complete, still not fastest) → dot product (fast, GPU-friendly matrix multiply) – this exact reasoning chain is a strong candidate for a “why dot-product attention?” essay question.
- **Zipf’s curve sketched explicitly** with the 3B words → 4 bytes → 12GB arithmetic (13/8).
- **Segmentation probability identity** $P(s_i, s_j) = P(s_j|s_i)P(s_i) = P(s_i|s_j)P(s_j) = \frac{P(s_i \cap s_j)}{P(s_i)P(s_j)}$ (13/8) – a compact way to justify frequency-based BPE merges probabilistically.
- **Byte Latent Transformer** (17/8): tokenisation-free approach treating fixed-size byte patches (e.g., 4 bytes) directly as tokens.
- **VC dimension** introduced early (6/8, “Neural Networks: Projects/Maps into another space/dimension – VC dimension”) well before its full treatment in Lecture 10 – shows the instructor circles back to reinforce concepts.
- **Research ⇒ Design ⇒ Engineering, underpinned by Abstraction and Understanding (Theory)** (10/8) – a meta-framing of how the field progresses that could appear as a conceptual/discussion question.

12 Seminar 1: Data Creation, Curation & Evaluation for LLMs

12.1 Overview

Slides: *ANLP-Seminar-Data-Creation.pdf*, based primarily on Long et al., “On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey” (Findings of ACL 2024). The seminar reframes the traditional “find data” paradigm (scrape, license, beg) as **data creation**: the deliberate design and construction of data for downstream tasks (most popularly, training models).

Why data creation, and the three-stage workflow

Why: data is expensive, often scarce in domains that matter most, and legally constrained. The field organises as a three-stage workflow: **Generation** (producing/synthesising candidate data) → **Curation** (deciding what to keep: filtering, quality scoring, reweighting) → **Evaluation** (checking whether it worked; optional if confident enough).

Formalising the generation process

$$\mathcal{D}_{\text{gen}} \leftarrow \mathcal{M}_p(\mathcal{T}, \mathcal{D}_{\text{sup}}), \quad p(\mathcal{T}, \mathcal{D}) \leftarrow E(e_{\text{task}}, e_{\text{condition}}, e_{\text{demo}})$$

$\mathcal{D}_{\text{gen}}$ is the final generated dataset; $\mathcal{M}_p$ is a pretrained model M prompted with p ; $\mathcal{T}$ is the task; $\mathcal{D}_{\text{sup}}$ is the seed data samples; the prompt p itself is composed from task, conditioning, and demonstration embeddings.

12.2 What Makes Good (Generated) Data?

Beyond obvious criteria (cleanliness, coverage, sufficient quantity, economic feasibility), two requirements dominate the literature:

- **Faithfulness**: data must be logically and grammatically coherent; hallucination and the fat-tailed knowledge distribution of LLMs introduce noise.
- **Diversity**: it is intractable to estimate the true underlying data-generating distribution for every real-world task and sample from it; LLMs’ inherent biases also cause monotonicity of outputs even at high decoding temperature.

12.3 Why Not Just Use the Internet?

- Common Crawl: 300 billion pages spanning 19 years, 3–5 billion new pages/month; essentially every open pretraining corpus (FineWeb, DCLM) is a filtered view of it.
- Research shows only a small percentage of the hundreds of trillions of tokens extractable from the internet survives a proven curation strategy.

DataComp-LM (DCLM) – the internet curation testbed

A testbed for controlled dataset experiments: DCLM-Pool is a standardised 240T-token Common Crawl corpus; after the filtering pipeline (heuristic cleaning → deduplication → model-based filtering with a FastText classifier), the DCLM-Baseline training set is only $\approx 3.8\text{T}$ tokens – **roughly 98% of the pool is discarded** before the model ever sees it.

The DCLM workflow: (A) select a compute scale → (B) build a dataset (filtering or mixing track) → (C) train a model with a fixed recipe → (D) evaluate on 53 downstream tasks (zero/few-shot), including MMLU (57-subject broad-knowledge benchmark).

12.4 Curation Is Subtractive Design: The FineWeb Pipeline

FineWeb’s 7-stage curation pipeline (“The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale”)

1. **Text extraction:** parse WARC files from Common Crawl with `trafilatura` – never blindly trust a pre-cleaned version.
2. **Language ID:** a classifier scores document language with a confidence threshold for English; URL blocklists also applied.
3. **Heuristic filters:** rules derived by comparing statistics of high- vs. low-quality data (e.g., fraction of lines ending in punctuation).
4. **PII removal:** a legal/ethical step – regex scrubbing of emails and IP addresses.
5. **MinHash deduplication:** near-duplicate removal (*not* a monotonic relation with quality – over-deduplication can hurt).
6. **Benchmark decontamination:** removing training documents that contain evaluation items, to preserve train/test separation.
7. **Model-based quality scoring:** e.g., Llama-3-70B-Instruct annotates sampled documents for educational value on a 0–5 scale, discarding low scores.

MinHash & the Jaccard Similarity Coefficient (a brief but exam-relevant detour)

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Example: $A = \{1, 2, 3, 4, 5, 6\}$, $B = \{1, 3, 5, 7\} \Rightarrow J(A, B) = |\{1, 3, 5\}| / |\{1, 2, 3, 4, 5, 6, 7\}| = 3/7$. **Problem:** with billions of documents, computing pairwise Jaccard similarity is too slow ($O(n^2)$ document pairs). **Solution – Locality Sensitive Hashing (LSH):** convert each document to a compact signature via a hash function H , small enough to fit in memory, such that if $\text{similarity}(d_1, d_2)$ is high then $P(H(d_1)=H(d_2))$ is high, and if similarity is low then that probability is low. A series of random permutations approximates the Jaccard coefficient efficiently, letting similar documents be clustered without exhaustive pairwise comparison.

12.5 From Curation to Synthesis: Two Stages of Generation

Generation splits across the two stages of the LLM pipeline: **pretraining** and **post-training**.

12.5.1 Synthesis #1 – Pretraining: WRAP (Web Rephrase Augmented Pretraining)

WRAP – “Rephrasing the Web” (Maini et al., ACL 2024)

Core idea: rewrite what you’d usually discard. Prompt an off-the-shelf instruct model (e.g., Mistral) to paraphrase noisy/unstructured web text into a target style (“like Wikipedia”, “Q&A format”), then pretrain on a mixture of real *and* synthetic (rephrased) data. **Result:** rephrased + real beats either alone, and gives a $\sim 3\times$ speed-up to reach the same zero-shot accuracy as training on $3\times$ more raw C4 data. **Why rephrased data has higher utility:** (1) it incorporates style diversity closer to downstream evaluation style; (2) it has higher signal density – raw web text is padded with boilerplate, typos, malformed structure, while a cleaned paraphrase carries the same content in fewer, better-formed tokens.

12.5.2 Synthesis #2 – Post-Training: Self-Instruct and Evol-Instruct

Self-Instruct (Wang et al., ACL 2023)

Core idea: you don’t have a corpus to rewrite, so grow one from a seed. Start with ~ 175 human-written seed tasks (1 instruction + 1 instance each) $\rightarrow$ prompt an LM to generate new instructions $\rightarrow$ classify whether the task is classification-type $\rightarrow$ generate instances (input-first or output-first, depending on classification) $\rightarrow$ filter low-quality/too-similar generations $\rightarrow$ add back to the task pool $\rightarrow$ repeat. The resulting instruction-output pairs are used to instruction-tune the LM itself.

Evol-Instruct (WizardLM, Xu et al., ICLR 2024)

Core idea: don’t sample brand-new tasks – **mutate existing ones** along complexity axes: add constraints, deepen, concretise, increase reasoning steps, complicate input (with a table/code/formula), or “in-breadth evolve” to a related but different task. Framed as “using LLMs instead of humans to automatically mass-produce open-domain instructions of various difficulty levels.”

12.6 The Student Inherits the Teacher (and Model Collapse)

The shared shape of every synthesis pipeline

WRAP has Mistral rephrasing text to train a 1.3B model; Self-Instruct has GPT-3 writing tasks; Evol-Instruct has ChatGPT writing responses that fine-tune LLaMA. **The pattern: a strong model writes the data for a weaker one.** The student inherits the teacher’s formatting, refusal style, and length distribution (mostly cosmetic) – but also two things that are **not** cosmetic: the **teacher’s blind spots** and the **teacher’s benchmark exposure**. The teacher is a ceiling: if your data is the teacher’s output, imitation caps you at the teacher.

Model Collapse (Shumailov et al., *Nature*, 2024)

Training recursively on model-generated content causes **irreversible defects**: the tails of the original content distribution disappear over successive generations. Three compounding error sources:

- **Statistical approximation error**: non-zero probability that information is lost at every resampling step; rare events get dropped by chance; vanishes as $n \rightarrow \infty$ but never at finite n .
- **Functional expressivity error**: the model class cannot represent the true distribution (e.g., fitting a Gaussian to bimodal data, or any finite network to an arbitrary density); negligible with sufficiently expressive models.
- **Functional approximation error**: the learning procedure itself is imperfect (SGD, regularisation, biased objectives) – even with infinite data and unlimited expressivity, the optimiser doesn’t land on the truth.

Symptom: probable events get overestimated, improbable events get underestimated; later generations produce text the original finds trivially predictable (tall narrow spikes in the perplexity histogram = a handful of sequences dominating).

Can we avoid model collapse? (Gerstgrasser et al., COLM 2024)

Instead of **replacing** old data with new synthetic data each generation, **accumulate** successive generations of synthetic data alongside the original real data. Empirically, accumulation avoids the monotonic test-loss increase seen under replacement (which rises with each model-fitting iteration); accumulation keeps test loss flat/decreasing across iterations, consistent with WRAP’s “mixture is better than the parts” finding.

12.7 Evaluation: Benchmark Decontamination

Why decontamination matters, and why it’s hard

Most decontamination is **string matching** (n-gram overlap); simple variations of test data (paraphrasing, translation) easily bypass this. **Embedding similarity** search detects more broadly but with many false positives, and it’s hard to find a good similarity threshold. A 13B model can trivially overfit a leaked test benchmark and match GPT-4-level scores.

LLM Decontaminator (two-step algorithm)

(1) For each test case, retrieve the top- k most similar training items via embedding similarity search. (2) An advanced LLM (e.g., GPT-4) judges each retrieved pair: is this training item a rephrased/paraphrased version of the test item? Returns True/False. This catches rephrased contamination that n-gram overlap and raw embedding similarity both miss (n-gram overlap has limited ability to spot rephrased samples; embedding similarity alone gives too many false positives).

Quantified contamination rates (from the seminar’s cited table)

Rephrased-sample contamination percentages found across training/benchmark pairs: The Stack (4G subset) on HumanEval $\approx 18.9\%$; StarCoder-Data (2.4G) on HumanEval $\approx 15.9\%$; MATHInstruct on MATH Test $\approx 15.4\%$; RedPajama-Data (16G) on HumanEval $\approx 8.5\%$. **Takeaway:** even “clean-looking” training sets can carry double-digit percentages of near-duplicate test items once paraphrase-aware detection is used.

12.8 Other Evaluation Methods (Short Notes)

Category	Method
Data Faithfulness	Ground-truth comparison (automatic scoring where reference labels exist); Human evaluation (accuracy scales with sample size and cost); Auxiliary model evaluation (a model replaces the expert – cheaper, broader coverage); Information extraction (pull relevant spans first so the judge model fits them in context).
Data Diversity	Vocabulary statistics (vocab size, n-gram frequency – intuitive but semantically blind); Sample relevance (cosine similarity, sample distance – captures semantic spread).

12.9 Open Problems (flagged by the seminar as unresolved)

1. **Semantic decontamination doesn’t scale:** an embedding index over a trillion-token corpus is affordable, but LLM-judging every retrieved candidate against every benchmark item is not.
2. **A bigger teacher doesn’t make better data:** scaling the generator does not monotonically help – 8B generators consistently beat 3B ones, but going up to 70B often produced *worse* synthetic data.
3. **No scaling law for synthetic data, and the literature disagrees on the mixing ratio:** a 1000-model study found optimal synthetic ratios converge near 30%, stable across sizes/budgets; other work finds the optimal ratio *rises* with model size ($\sim 12.5\%$ at 3B, $\sim 25\%$ at 6B); WRAP used 1:1. No formal theory yet.

12.10 Practice & Review

Q1. [MC] According to the DCLM pipeline, approximately what fraction of the DCLM-Pool (Common Crawl) is discarded before the model ever sees it?

- (A) $\sim 10\%$
- (B) $\sim 50\%$
- (C) $\sim 98\%$
- (D) $\sim 0\%$ (nothing is discarded)

Answer: (C). DCLM-Pool is 240T tokens; the filtered DCLM-Baseline is only ≈ 3.8 T tokens.

- Q2. [Short Answer]** Compute the Jaccard similarity of $A = \{1, 2, 3, 4, 5, 6\}$ and $B = \{1, 3, 5, 7\}$, and explain why Locality Sensitive Hashing is used instead of computing this directly at web scale. *Answer:* $J(A, B) = |A \cap B| / |A \cup B| = |\{1, 3, 5\}| / |\{1, 2, 3, 4, 5, 6, 7\}| = 3/7$. Computing pairwise Jaccard similarity directly across billions of documents requires $O(n^2)$ comparisons, which is intractable; LSH instead hashes each document to a compact signature such that similar documents collide (are hashed to the same bucket) with high probability, letting near-duplicates be found without exhaustive pairwise comparison.
- Q3. [Short Answer]** Why does WRAP’s rephrased synthetic data have “higher utility” (fewer tokens needed) than the equivalent amount of raw web text? *Answer:* Raw web text is padded with boilerplate, typos, and malformed structure, diluting useful signal per token. A rephrased paraphrase carries the same underlying content in a cleaner, more information-dense form, and can also be steered toward a target style (e.g., Wikipedia-like) that better matches downstream evaluation formats, so the model needs to see fewer tokens to extract the same amount of useful signal.
- Q4. [Short Answer]** List and briefly explain the three error sources that Shumailov et al. identify as causing model collapse. *Answer:* (1) Statistical approximation error – finite sampling at each generation has a non-zero chance of losing rare information, vanishing only as sample size $\rightarrow \infty$. (2) Functional expressivity error – the model class itself cannot perfectly represent the true data distribution (e.g., a Gaussian fit to bimodal data), independent of sample size. (3) Functional approximation error – the learning procedure (SGD, regularisation, biased objectives) is imperfect, so even with infinite, perfectly expressive capacity the optimiser would not recover the true distribution exactly.
- Q5. [Essay]** Describe the FineWeb curation pipeline stage by stage, and explain why deduplication is described as “not a monotonic relation with quality.” *Answer:* The pipeline runs: text extraction (parsing WARC files with *trafilatura*, since pre-cleaned versions cannot be trusted blindly) $\rightarrow$ language ID (confidence-thresholded classifier plus URL blocklists) $\rightarrow$ heuristic filters (rules derived from comparing high- vs. low-quality document statistics) $\rightarrow$ PII removal (regex scrubbing of emails/IPs for legal/ethical compliance) $\rightarrow$ MinHash deduplication (near-duplicate removal) $\rightarrow$ benchmark decontamination (removing documents overlapping with evaluation sets) $\rightarrow$ model-based quality scoring (an LLM annotates educational value on a 0–5 scale, discarding low scores). Deduplication is not monotonic with quality because removing *too many* near-duplicates can also remove naturally recurring, high-quality patterns or reduce coverage of legitimately common (and useful) content; more deduplication is not automatically better, so the degree of deduplication itself must be tuned rather than maximised.
- Q6. [Essay]** Compare the “replace” and “accumulate” strategies for handling successive generations of model-generated training data, and explain which one mitigates model collapse and why. *Answer:* Under the “replace” strategy, each new generation’s model is trained purely on the *previous* generation’s model-generated data, discarding the original real data entirely; empirically this causes test loss (cross-entropy) to increase monotonically across model-fitting iterations, the classic model-collapse signature, because errors and distributional narrowing compound with each resampling step. Under the “accumulate” strategy, each new generation’s training set is the *union* of all previous real and synthetic data (Data 1 + Data 2 + ... + Data n), so the original real distribution’s tails are never fully lost even as synthetic data is added. Gerstgrasser et al. show empirically that accumulation keeps test loss flat or decreasing across iterations rather than rising, because the ever-present real data continually anchors the

training distribution and prevents the progressive tail-narrowing that causes collapse under pure replacement.

13 Seminar 2: Reasoning in Language Models

13.1 Overview

Slides: *ANLP-Seminar-Reasoning.pdf*. Covers what reasoning is and why it’s needed, methods for eliciting/training reasoning (CoT, SFT-on-traces/StaR, and a full RL stack: REINFORCE → TRPO → PPO → RLHF → RLOO → GRPO), efficient reasoning (thinking budgets), and recent advances. **This is the single most formula-dense section of the syllabus** – expect derivation-style and “compare method X vs. Y” questions here.

What is reasoning, and why do we need it?

Reasoning: the ability to solve a problem or navigate a scenario by piecing together prior knowledge and facts. **Why:** humans solve complex multi-step problems (e.g., math) by reasoning through intermediate steps before the final answer; Wei et al. (2022) showed eliciting such intermediate steps in LMs greatly improves performance on complex reasoning tasks.

13.2 Method 1: Chain of Thought (CoT) – Wei et al., NeurIPS 2022

A few worked exemplars showing intermediate reasoning steps (not just question→answer) dramatically improve accuracy on arithmetic/commonsense/symbolic reasoning benchmarks (GSM8K, SVAMP, MAWPS).

CoT is an emergent ability of scale

Standard prompting on GSM8K-style problems often fails even when the model “knows” the sub-facts; CoT prompting fixes this by forcing explicit intermediate computation. Critically, the CoT accuracy *gain over standard prompting only appears at sufficient model scale* (LaMDA, GPT, PaLM curves all show CoT diverging from standard prompting only past a scale threshold) – below that threshold, CoT can even hurt performance slightly.

Models can fail when surface patterns are preserved but problem-solving requirements change (MATH-Perturb, Huang et al. 2025)

Simple perturbations (numerical changes, asking for a related quantity, adding/removing non-essential constraints) preserve the *same* solving method. **Hard perturbations** (increasing mathematical complexity, enlarging key numbers to disable brute force, relaxing constraints, removing simplifying properties) require a *different* solving method and often change the answer. Reasoning-mode models (Gemini-2.0-flash-thinking, o1-preview, o1-mini) degrade far less under hard perturbation ($\Delta\text{All} \approx 14\text{--}16$ points) than non-reasoning models (GPT-4o, Claude-3-Opus: $\Delta\text{All} \approx 22\text{--}28$ points) – concrete evidence that explicit reasoning generalises better under distribution shift in problem structure.

13.3 Method 2: Supervised Fine-Tuning on Reasoning Traces – STaR (Zelikman et al., 2022)

STaR – Self-Taught Reasoner

Instead of eliciting reasoning purely via prompting, **train** the model on it: generate multiple reasoning traces per question, keep only those reaching the **correct** final answer, and fine-tune on those. **Problem:** this loop eventually stalls on questions the model can never solve (zero correct traces $\Rightarrow$ no training signal). **Fix – rationalization:** for questions the model got wrong, feed it the *correct answer as a hint* and ask it to generate a rationale justifying that answer backward; fine-tune on these rationalized traces too.

Method	GSM8K Test Acc. (%)	Train Data Used (%)
Few-shot Direct GPT-J	3.0	~ 0
Few-shot CoT GPT-J	3.1	~ 0
GPT-J Direct Finetuned	5.8	100
STaR without rationalization	10.1	25.0
STaR with rationalization	10.7	28.7

STaR beats *full* direct fine-tuning (5.8%) while using only $\sim 29\%$ of the training data – evidence that *how* you fine-tune (on self-generated correct reasoning, with rationalization for hard cases) matters more than raw data volume.

13.4 Method 3: Reinforcement Learning (RL) – The Full Stack

Foundational definitions

Law of Effect (Thorndike, 1911): “The greater the satisfaction or discomfort, the greater the strengthening or weakening of the bond.” **RL** (Sutton & Barto): learning what to do – how to map situations to actions – so as to maximize a numerical reward signal. Key tension: **Explore vs. Exploit.**

Element	General meaning	For Language Models
Agent π_θ	Learner/decision-maker maximising expected cumulative reward	The LM with parameters θ , generating tokens per policy π_θ
Environment $\mathcal{E}$	External system the agent interacts with	Everything outside the agent that responds to its actions
State s_t	Current situation the agent observes	Current prompt + tokens generated so far
Action a_t	A choice made per the policy	Generating the next token
Policy $\pi_\theta(a s)$	How the agent chooses an action given a state	LM's next-token probability distribution P_θ
Reward r_t/R	Scalar feedback on how good/bad the action was	$R(x, y)$: scalar feedback on the generated response/trajectory
Trajectory τ	Sequence of states, actions & rewards in one interaction	$\tau = \{s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T\}$ = prompt + response + rewards

The RL objective

$$\pi^* = \arg \max_{\pi} J(\pi), \quad J(\pi) = \int_{\tau} P(\tau | \pi) R(\tau) = \mathbb{E}_{\tau \sim P(\tau | \pi)} [R(\tau)]$$

$$P(\tau | \pi) = p_0(s_0) \prod_{t=1}^T P(s_t | s_{t-1}, a_{t-1}) \pi(a_{t-1} | s_{t-1})$$

The expectation is over *all possible trajectories*: $|\tau| = |\mathcal{V}|^N$ – combinatorially non-tractable, motivating Monte-Carlo estimation.

Action-Value, State-Value, and Advantage

$$A_{\pi_\theta}(s, a) = \underbrace{Q_{\pi_\theta}(s, a)}_{\text{Action-Value}} - \underbrace{V_{\pi_\theta}(s)}_{\text{State-Value}}$$

$Q_{\pi_\theta}(s, a)$: expected cumulative reward starting in state s , taking action a , then following π_θ . $V_{\pi_\theta}(s)$: expected cumulative reward starting in state s (averaged over the policy's own action choice). $A_{\pi_\theta}(s, a)$: how much better/worse action a is than the policy's expected performance from s – **the central quantity every method below tries to estimate cheaply**.

13.4.1 REINFORCE (Williams, 1992)

REINFORCE – Monte-Carlo policy gradient

$$\nabla_{\theta} J(\pi_{\theta_0}) \approx \frac{1}{|N|} \sum_{\tau \in N} \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta_0}(a_{t-1} | s_{t-1}) R(\tau)$$

Sample a set N of trajectories via Monte-Carlo and estimate the gradient directly from the raw trajectory reward $R(\tau)$ (no baseline).

Why REINFORCE alone is not enough

High variance: scaling every action's gradient by the *entire* trajectory's reward means a single noisy/lucky trajectory can swing all its action-gradients together, and different trajectories of very different quality can produce wildly different gradient estimates.

13.4.2 TRPO (Schulman et al., ICML 2015)

Trust Region Policy Optimisation

Importance ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$.

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^T r_t(\theta) A_{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}_t [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t))] \leq \delta$$

TRPO fixes REINFORCE's per-action credit assignment (via the importance ratio and an explicit advantage A) and its instability (via the hard KL-divergence trust-region constraint bounding how far the new policy can move from the old one) – but the constrained optimisation is expensive to solve exactly.

13.4.3 PPO (Schulman et al., 2017)

PPO's key simplification

Drop the hard KL constraint; instead **clip** the importance ratio directly inside the objective, discouraging updates that move $r_t(\theta)$ too far from 1.

PPO-Clip objective

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^T \min \left(r_t(\theta) A_{\pi_{\theta_{\text{old}}}}(s_t, a_t), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right) \right]$$

The min of the unclipped and clipped terms means: if the advantage is positive, gains from increasing r_t beyond $1 + \epsilon$ are capped; if negative, the penalty from decreasing r_t below $1 - \epsilon$ is capped – either way, large policy updates in a single step are discouraged *without* an explicit KL constraint.

Full PPO training objective (as used for LLMs)

$$J_{\text{PPO}}(\theta, \gamma) = \underbrace{\mathbb{E}[\min(\cdot, \cdot)]}_{\text{Maximise reward}} + \underbrace{w_1 H(\pi_\theta)}_{\text{Maximise entropy}} - \underbrace{w_2 \mathbb{E}_t[D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}})]}_{\text{Penalise KL divergence}} - \underbrace{w_3 \mathcal{L}_V(\gamma)}_{\text{Critic L2}}$$

Critic as baseline: $A(x, y) = R(x, y) - V(x)$ – a learned value model V estimates the expected reward, and the advantage is the reward’s deviation from that estimate (reduces variance vs. raw REINFORCE, without needing a full trust-region constraint).

13.4.4 RLHF (Ouyang et al., InstructGPT, 2022)

Why RLHF: cross-entropy/BLEU/ROUGE can’t capture human preference

Can we use human feedback on generated text as a performance measure, then use that feedback as a reward signal to optimise the LM? **Three-step pipeline:** (1) Collect human demonstrations, train a supervised policy via SFT. (2) Collect comparison data (rank multiple model outputs best-to-worst); train a **Reward Model (RM)** on these rankings. (3) Optimise the SFT policy against the RM using RL (PPO).

Reward Model training objective, derived from the Bradley-Terry model

$$P(y_i \succ y_j | x) = \frac{\exp(R_\theta(x, y_i))}{\exp(R_\theta(x, y_i)) + \exp(R_\theta(x, y_j))}, \quad \mathcal{L}(\theta) = -\log \sigma(R_\theta(x, y_i) - R_\theta(x, y_j))$$

$y_i \succ y_j$ means a human labeler preferred response y_i over y_j ; the loss pushes the reward model to score the preferred response higher.

RLHF RL-phase training objective

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} \left[R_\phi(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right] + \gamma \mathbb{E}_{x \sim \mathcal{D}_{\text{pretrain}}} [\log \pi_\theta(x)]$$

The β term is a KL penalty against the original SFT/reference policy π_{ref} (prevents reward over-optimisation/degenerate outputs); the γ term mixes in the original pretraining loss to avoid regressing on general language ability.

Result

Human evaluators consistently preferred PPO(-ptx) > SFT > prompted-GPT > raw GPT, with the win-rate gap over SFT-175B **widening** as model size increases (1.3B → 175B) – RLHF gains compound with scale.

13.4.5 RLOO (Kool, van Hoof & Welling, ICLR 2019 workshop)

RLOO – “Buy 4 REINFORCE Samples, Get a Baseline for Free”

Reduces the variance of the policy-gradient estimator **without training a separate critic**, by using the rewards of the *other* sampled responses (for the same prompt) as a leave-one-out baseline.

Leave-One-Out advantage

$$A(x, y_i) = R(x, y_i) - \frac{1}{K-1} \sum_{\substack{j=1 \\ j \neq i}}^K R(x, y_j) = \frac{K}{K-1} \left(R(x, y_i) - \frac{1}{K} \sum_{j=1}^K R(x, y_j) \right)$$

Sample K responses per prompt; each response’s advantage is its reward minus the mean reward of the *other* $K-1$ responses. Plugged into the same REINFORCE gradient: $\nabla_{\theta} J \approx \frac{1}{|N|} \sum_{\tau \in N} \sum_t \nabla_{\theta} \log \pi_{\theta_0}(a_{t-1}|s_{t-1}) A(\tau)$.

13.4.6 GRPO (Shao et al., DeepSeekMath, 2024)

GRPO – Group Relative Policy Optimisation: PPO without a critic

PPO requires a separate critic/value model to estimate $V(s)$. GRPO eliminates the critic entirely: sample a **group** of G responses for the same prompt, and use their **group-relative rewards** (normalised by the group’s own mean and std) as the advantage.

GRPO advantage and objective

$$A_i = \frac{R_i - \mu_R}{\sigma_R}, \quad \mu_R = \frac{1}{G} \sum_{j=1}^G R_j, \quad \sigma_R = \sqrt{\frac{1}{G} \sum_{j=1}^G (R_j - \mu_R)^2}$$

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t}) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right]$$

where $r_{i,t}(\theta) = \pi_{\theta}(y_{i,t}|x, y_{i,<t}) / \pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})$ (same PPO-style clipped importance ratio, now per-token within each group member), and the KL term uses an unbiased estimator $D_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] = \frac{\pi_{\text{ref}}}{\pi_{\theta}} - \log \frac{\pi_{\text{ref}}}{\pi_{\theta}} - 1$ evaluated per-token.

GRPO in DeepSeek-R1: rule-based rewards + multi-stage pipeline

$\text{Reward}_{\text{rule}} = \text{Reward}_{\text{acc}} + \text{Reward}_{\text{format}}$ (correctness + adherence to an output format, e.g., `<think>...</think>`) – a **rule-based** reward avoids training/gaming a learned reward model for verifiable domains like math. DeepSeek-R1’s pipeline: DeepSeek-V3-Base → RL (rule-based reward on reasoning prompts) → **R1-Zero** → sample & filter/refine → SFT “cold start” on long CoT → R1-Dev-1 → RL with rule-based reward + language-consistency reward → R1-Dev-2 → SFT on mixed reasoning/non-reasoning data → R1-Dev-3 → RL

with rule-based + preference reward → **R1**. An intermediate R1-Zero checkpoint exhibited a documented “**aha moment**”: mid-solution, the model spontaneously writes “Wait, wait. Wait. That’s an aha moment I can flag here” and backtracks to re-derive its work – self-correction emerging purely from RL, without being explicitly trained to say this.

13.5 Efficient Reasoning: Thinking Budgets

Why efficient reasoning matters

Reasoning is expensive (long chains of thought cost tokens/latency); reasoning effort should scale with task difficulty, not be fixed.

- **Qwen3** (Yang et al., 2025): **Thinking Control** – integrates both a “thinking” mode (`/think`, wraps a `<think>...</think>` block before the response) and a “non-thinking” mode (`/no_think`, empty think block) in one model via SFT data formatted for each, plus a 4-stage post-training pipeline (Long-CoT cold start → Reasoning RL → Thinking-mode fusion → General RL) and strong-to-weak distillation for lightweight variants. **Thinking Budget**: the model can also handle *incomplete* thinking, generating a response based on however much reasoning fit in a token budget.
- **Nemotron Nano 2** (NVIDIA, 2025): explicit thinking budget via a multi-stage post-training pipeline (SFT1 → SFT2 → SFT3 → [DPO, GRPO→RLHF] → Merged), with the model specifically **trained to tolerate truncation** of its reasoning trace – accuracy on AIME25/GPQA/LiveCodeBench/MATH500 rises smoothly as the token budget increases from 0 to 32k, and after truncation-tolerance training, accuracy no longer collapses when reasoning is cut short.

13.6 Recent Advances (Pointers)

- **Agentic Reasoning**: foundational capabilities, self-evolving adaptation, and collective (multi-agent) coordination (Wei et al., 2026).
- **Multimodal Reasoning**: extending chain-of-thought-style reasoning beyond text to vision/other modalities (Lin et al., 2025).
- **Latent Reasoning via Looped Transformers**: instead of reasoning in natural-language tokens, a fixed set of layers is applied *repeatedly* ($t = 1, \dots, T_{\max}$) to refine a latent representation, with an **exit gate** at each loop iteration deciding whether to stop early (via a CDF-thresholded early-exit criterion at inference) – amortises reasoning depth adaptively without emitting explicit CoT tokens (Dehghani et al. 2018 “Universal Transformers”; Fan et al. 2026; Zhu et al. 2025, “Ouro”).

13.7 Practice & Review

Q1. [MC] What is the key difference between TRPO and PPO?

- (A) PPO uses a hard KL-divergence constraint; TRPO clips the importance ratio
- (B) TRPO uses a hard KL-divergence trust-region constraint; PPO instead clips the importance ratio directly in the objective

- (C) They are mathematically identical
- (D) Neither uses an importance ratio

Answer: (B).

Q2. [MC] In GRPO, the advantage for response i in a group of G sampled responses is computed as:

- (A) $R_i - V(x)$ using a learned critic
- (B) $(R_i - \mu_R)/\sigma_R$, the group-normalised reward
- (C) The raw reward R_i with no baseline
- (D) The leave-one-out mean of the other $G - 1$ rewards subtracted, without normalising by std

Answer: (B).

Q3. [Short Answer] Why does REINFORCE suffer from high variance, and name two later methods that address this in different ways. *Answer:* REINFORCE scales the gradient of *every* action in a trajectory by that trajectory’s single overall reward $R(\tau)$, so noisy or unusually high/low-reward trajectories produce disproportionately large or small gradient updates across all their actions, causing high variance in the gradient estimate across sampled trajectories. PPO addresses this with a learned critic baseline ($A = R - V(x)$) and a clipped objective limiting update size; RLOO/GRPO address it without any critic, by using the rewards of other sampled responses to the same prompt as a baseline (leave-one-out mean for RLOO, group mean/std normalisation for GRPO).

Q4. [Short Answer] State the Bradley-Terry-derived reward model loss used in RLHF, and explain what $y_i \succ y_j$ means in this context. *Answer:* $\mathcal{L}(\theta) = -\log \sigma(R_\theta(x, y_i) - R_\theta(x, y_j))$. Here $y_i \succ y_j$ denotes that a human labeler judged response y_i as preferable to response y_j for the same prompt x ; minimising this loss increases the reward model’s score gap in favour of the preferred response, consistent with the Bradley-Terry pairwise-comparison probability model.

Q5. [Short Answer] What problem does STaR’s “rationalization” step solve, and how does it work? *Answer:* Without rationalization, STaR’s self-training loop only produces training signal for questions the model can already answer correctly (since only correct-answer traces are kept), so it eventually stops improving on questions it consistently gets wrong. Rationalization solves this by, for questions the model got wrong, revealing the correct answer as a hint and asking the model to generate a rationale that justifies that answer, then fine-tuning on these rationalized traces as well – giving the model a training signal even for problems it could not originally solve unaided.

Q6. [Essay] Trace the evolution of policy-gradient methods for LLM post-training from REINFORCE through PPO to GRPO, explaining what problem each new method solves relative to its predecessor. *Answer:* **REINFORCE** provides the basic Monte-Carlo policy-gradient estimator, $\nabla_\theta J \approx \frac{1}{|N|} \sum_\tau \sum_t \nabla_\theta \log \pi_\theta(a_{t-1}|s_{t-1})R(\tau)$, but suffers from high variance because every action in a trajectory is scaled by the same raw trajectory reward with no baseline. **TRPO** introduces per-step credit assignment via an explicit advantage $A_{\pi_{\theta_{\text{old}}}}(s, a)$ and an importance ratio $r_t(\theta)$, while constraining the policy update to a KL-divergence trust region

to prevent destructively large updates – but the constrained optimisation is computationally expensive to solve exactly at each step. **PPO** keeps the importance-ratio/advantage machinery but replaces the hard KL constraint with a simple clipped objective (min of the unclipped and clipped importance-weighted advantage), which is far cheaper to optimise via standard SGD while still discouraging overly large policy updates; PPO also typically uses a learned critic $V(x)$ as the advantage baseline, which itself must be trained and can be a source of instability or extra compute. **GRPO** removes the critic entirely: for a given prompt, it samples a *group* of G responses and computes each one’s advantage by normalising its reward against the group’s own mean and standard deviation ($A_i = (R_i - \mu_R)/\sigma_R$), reusing PPO’s clipped objective per-token within each group member. This eliminates the cost and instability of training a separate value model while still providing a low-variance, self-normalising baseline, making GRPO particularly well suited to LLM reasoning tasks (as in DeepSeekMath and DeepSeek-R1) where many responses per prompt can cheaply be sampled and rule-based rewards (e.g., correctness on math problems) are available.

- Q7. [Essay]** Explain the three-step RLHF pipeline (Ouyang et al., 2022) and the role of the KL penalty β and pretraining-mixture term γ in the final RL objective. *Answer:* RLHF proceeds in three steps: (1) collect human demonstrations of desired behaviour and use them to train a supervised (SFT) policy via standard fine-tuning; (2) collect comparison data by sampling multiple outputs from the SFT policy for a prompt and having human labelers rank them from best to worst, then train a reward model (RM) to predict these preferences via the Bradley-Terry-derived loss $-\log \sigma(R_\theta(x, y_i) - R_\theta(x, y_j))$; (3) optimise the SFT policy against the frozen RM using PPO, sampling a new prompt, generating an output, scoring it with the RM, and updating the policy via the reward. The final RL objective maximises $\mathbb{E}[R_\phi(x, y)] - \beta \mathbb{E}[\log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))] + \gamma \mathbb{E}_{\text{pretrain}}[\log \pi_\theta(x)]$. The β -weighted KL term penalises the policy for straying too far from the original reference (SFT) policy, preventing reward-model over-optimisation/degenerate high-reward-but-low-quality outputs (reward hacking); the γ -weighted pretraining term mixes in the original language-modelling loss on pretraining data, preventing the RL fine-tuning phase from catastrophically forgetting general language capabilities while it optimises for the narrower reward signal.

14 Previous-Year Question Patterns & Predictions

14.1 2023 Mid-Sem (M23CS7.501, Sep 22 2023) – 30 marks, 1.5 hrs

1. Explain how RNNs enable handling long-distance dependencies. Compare LSTMs and GRUs architecturally. [5+5]
2. Describe the ELMo contextual embedding model. How is it used for downstream tasks? [5+5]
3. Describe a seq2seq MT model where the decoder attends to the encoder. Why is attention needed? [3+7]

Grading notes from the answer key (handwritten annotations): Q1 wanted *recurrence AND back-propagation-through-time* discussed, not just gate mechanics; explicitly noted “having more gates is not that important, similarity [between LSTM/GRU] matters more” (GRU: 2 gates, no internal memory cell, fewer params, faster, less memory vs. LSTM: 3 gates, computationally slower but more expressive). Q2 wanted: ELMo = stacked Bi-LSTM; **weighted sum of layer embeddings**, with weights learned/computed based on the downstream task. Q3 wanted: describe RNN/LSTM with attention (“fixed”) defined; note that *different target words need different content words* from the source for translation.

14.2 2024 Mid-Sem (M24CS7.501, Sep 23 2024) – 30 marks, 1.5 hrs

1. Explain how we can enable handling long-distance dependencies in RNNs. [5]
2. Why is attention needed in a sequence-to-sequence model? Explain with the attention function description. [5]
3. Describe the transformer encoder block in detail. [10]
4. How does LoRA fine-tuning differ from Adapters? Explain with architectures. [10]

14.3 Cross-Year Pattern Recognition

Recurring theme	2023	2024
RNN long-distance dependency handling	Q1	Q1
Why attention is needed in seq2seq	Q3 (framed via MT)	Q2 (framed directly)
Architecture description “in detail”	Q2 (ELMo)	Q3 (Transformer encoder)
Fine-tuning / adaptation methods	–	Q4 (LoRA vs. Adapters)

What this predicts for the 2026 quiz/exam, given this year’s syllabus emphasis

- **Very likely:** A “describe the Transformer encoder (and/or decoder) block in detail” essay – this appeared verbatim in 2024 and this year’s lectures cover it exhaustively (Lecture 2). Practice writing the full block flow from memory: embedding+PE → MHA → Add&Norm → FFN → Add&Norm, and the decoder’s extra masked self-

attention + cross-attention.

- **Very likely:** “Why is attention needed?” / “explain the attention function” – master the scaled dot-product formula, the Q/K/V roles, and the fixed-length-bottleneck motivation from Lecture 2/3.
- **Likely, updated for this year’s syllabus:** Given the heavy 2026 emphasis on **positional encoding variants** (Lecture 6) and **efficient attention / KV-cache / MQA-GQA-MLA** (Lecture 7) and **normalization placement** (Lectures 8–9) – these are new, heavily-slide-supported topics this year (absent from the RNN/LSTM-centric 2023/2024 papers) and are a natural place for this year’s exam to differentiate itself, since the course has clearly shifted from “RNN/attention basics” toward “modern efficient-Transformer engineering.”
- **Possible gap:** LoRA vs. Adapters (2024 Q4) was **only briefly foreshadowed** in this year’s Lecture 4 materials (LoRA numeric example) with no explicit Adapters lecture content found through 17 Sept. If this pattern repeats, revise the LoRA/Adapters comparison box in Lecture 4 as a hedge.
- **Likely:** BPE/tokenization details (Zipf’s law, vocabulary-size math, BPE algorithm steps) given how much slide/lecture time (Lecture 5) was devoted to it – a strong candidate for a numeric/derivation-style question analogous to the parameter-counting exercises in the slides.
- Expect **multi-part questions with explicit mark allocations** (this instructor consistently splits multi-clause questions into sub-marks, e.g., “[5+5]” or “[3+7]”) – practice answering each clause of a compound question explicitly and completely, since partial marks are clearly awarded per clause.

15 Quick-Reference Formula Sheet

Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad \text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad c_i = \sum_j \alpha_{ij} V_j$$

Causal mask: $M_{ij} = 0$ if $i \geq j$, else $-\infty$, added to scores pre-softmax.

Positional Encoding

Sinusoidal: $PE_{(p,2i)} = \sin(p/10000^{2i/d})$, $PE_{(p,2i+1)} = \cos(p/10000^{2i/d})$.

RoPE: rotate (x, y) pairs of Q, K by $\theta_p = p\theta$; $Q' \cdot K' \propto \cos(\theta_i - \theta_j)$ – pure relative-distance dependence, unbounded extensibility.

Relative PE: add learned bias b_Δ ($\Delta = i - j$) directly to attention score, capped table (e.g., ± 128).

Tokenization

Zipf: $f(r) \approx C/r^s$, $s \approx 1 \Rightarrow$ top $\sim 3\text{--}5\%$ of types cover $\sim 95\%$ of tokens.

Embedding params = $|V| \times d$. BPE: repeatedly merge most-frequent adjacent symbol pair until target vocab size.

Efficient Attention / KV-Cache (per-token cache size)

MHA: $2n_h d_h l$ MQA: $2d_h l$ GQA: $2n_g d_h l$ MLA: $(d_c + d_h^R)l \approx \frac{9}{2}d_h l$

Linear attention (causal/RNN form): $S_t = S_{t-1} + \phi(k_t)v_t^\top$, $z_t = z_{t-1} + \phi(k_t)$, $o_t = \phi(q_t)^\top S_t / \phi(q_t)^\top z_t - O(1)$ per token, no KV cache.

Normalization

BatchNorm: $\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, stats over batch.

LayerNorm: $\hat{x}_j = \frac{x_j - \mu_\ell}{\sqrt{\sigma_\ell^2 + \epsilon}}$, stats over features (per sample).

RMSNorm: $\hat{y}_i = \gamma x_i / (r + \epsilon)$, $r = \sqrt{\frac{1}{n} \sum x_i^2}$ (LayerNorm with $\mu, \beta = 0$).

Pre-Norm: $x + F(\text{LN}(x))$. Post-Norm: $\text{LN}(x + F(x))$. DeepNorm: $\text{LN}(\alpha x + F(x))$.

Neural Net Foundations

Perceptron: $y = \text{sign}(w \cdot x + b)$; update $w \leftarrow w + \eta(y - \hat{y})x$. VC-dim (linear, $\mathbb{R}^d$) = $d + 1$.

FFN: $\max(0, xW_1 + b_1)W_2 + b_2$. GLU: $\sigma(xW) \otimes xV$. SwiGLU: $\text{Swish}_\beta(xW) \otimes xV$ (3 matrices)

$\Rightarrow$ shrink d_{ff} by $\frac{2}{3}$).

MoE: $H(x) = xW$, $G(x) = \text{softmax}(H(x))$, $y = \sum_e G(x)_e E_e(x)$, route to top- k . Load-balance: $L_{\text{bal}} = w \cdot \text{CV}$, $\text{CV} = \sigma/\mu$. Expert capacity = (tokens/batch/num_experts) $\times$ capacity_factor.

16 Rapid Self-Test Bank (Answer Key at End)

Attempt all before checking answers. Mix of MC/short-answer spanning all 10 lectures.

1. Why can't RNNs be parallelised across the time dimension during training?
2. What does the $\sqrt{d_k}$ term in scaled dot-product attention prevent?
3. In cross-attention, which side provides Q, and which provides K/V?
4. Name the component that supplies non-linearity in a Transformer block.
5. What fraction of vocabulary types typically covers $\sim 95\%$ of tokens (Zipf)?
6. List the 5 steps of the BPE algorithm.
7. What is SentencePiece's key trick that makes it language-agnostic?
8. State the RoPE rotation formula for a 2-D sub-vector.
9. Why is RoPE "infinitely extensible" but a capped relative-position table is not?
10. What is the purpose of a KV cache, and when is it used (training vs. inference)?
11. Rank MHA, GQA, MQA, MLA by KV-cache size (largest to smallest).
12. Why can GQA be derived from an existing MHA checkpoint but MLA cannot?
13. What does linear attention replace inside the softmax to achieve linear time complexity?
14. Give two reasons plain linear attention underperforms softmax attention.
15. Contrast BatchNorm and LayerNorm on what population they normalise over.
16. RMSNorm is LayerNorm with which two quantities forced to zero?
17. Contrast Pre-Norm and Post-Norm on where LayerNorm sits relative to the residual add.
18. What diagnostic (a specific similarity metric) reveals representation collapse?
19. What is the VC-dimension of linear separators in $\mathbb{R}^d$?
20. What does a GLU compute, and why does gating help training efficiency?
21. In MoE, what does the auxiliary load-balancing loss prevent?
22. Define Pidgin vs. Creole.
23. Why was the dot product chosen over cosine similarity or an MLP for attention scoring?
24. What historical event (1969) triggered an "AI winter," and why?
25. Give the formula for embedding-layer parameter count in terms of $|V|$ and d .

Answer Key

1. Each hidden state depends recursively on the previous one, forcing sequential computation.
2. It prevents the dot products from growing large in magnitude and saturating the softmax (which would cause vanishing gradients).
3. Decoder provides Q; encoder's final layer output provides K and V.
4. The position-wise Feed-Forward Network (FFN).
5. $\sim 3\text{--}5\%$.
6. Init vocab = chars/bytes; count adjacent pair frequencies; merge most frequent pair; replace occurrences; repeat until target vocab size.
7. It treats the space character as an ordinary symbol, needing no language-specific pre-tokenisation.
8. $(x', y') = (x \cos \theta_p - y \sin \theta_p, x \sin \theta_p + y \cos \theta_p)$, $\theta_p = p\theta$.
9. RoPE's relative-angle dependence is a continuous mathematical function of position (works for any p); a capped table has zero bias beyond its max distance, hard-limiting range.
10. Avoids recomputing K,V for all previous tokens at each generation step; used only at inference, not training (training processes the full sequence in parallel).
11. $\text{MHA} > \text{GQA} > \text{MQA} \approx \text{MLA}$ is close, but per the DeepSeek table: $\text{MHA} > \text{GQA} > \text{MLA} \approx \text{MQA}$ -ish, precisely: $\text{MHA} (2n_h d_h l) > \text{GQA} (2n_g d_h l) > \text{MLA} (\approx 4.5 d_h l) > \text{MQA} (2d_h l)$ numerically can vary by config, but conceptually MHA is largest and MQA/MLA are smallest, with GQA in between.
12. GQA's shared K/V per group can be obtained by mean-pooling existing per-head K/V projections from an MHA checkpoint; MLA's jointly-compressed low-rank latent has no structural counterpart in MHA and must be pre-trained from scratch.
13. Replaces $\exp(q^\top k)$ with a factorisable kernel $\phi(q)^\top \phi(k)$, letting the key/value sum be pre-computed once independent of the query.
14. (Any two of) bounded fixed-size recurrent state degrades long-range retrieval; smoother/less sharp weighting than softmax (attention dilution); no forgetting mechanism (pure accumulation).
15. BatchNorm: across the batch, per feature. LayerNorm: across features, per sample.
16. Mean μ and shift parameter β .
17. Pre-Norm: LayerNorm before the sub-layer (sub-layer sees normalised input). Post-Norm: LayerNorm after the residual addition.
18. Cosine similarity between hidden states of consecutive layers, $\cos(h_\ell, h_{\ell+1})$.
19. $d + 1$.

20. $a \otimes \sigma(b)$ – gating suppresses irrelevant dimensions, improving gradient signal-to-noise and training efficiency.
21. Expert collapse (all tokens routing to only a few experts, starving the rest of training signal).
22. Pidgin: simplified contact language, no native speakers; Creole: a pidgin that gained native speakers and a full grammar.
23. Dot product is expressible as a single GPU-efficient matrix multiplication ($QK^\top$); cosine similarity needs extra norm computations, and an MLP cannot be batched into one matrix multiply at all.
24. Minsky & Papert’s *Perceptrons* proved a single perceptron cannot solve XOR (not linearly separable), stalling connectionist research until back-propagation (1986) enabled trainable multilayer (non-linear) networks.
25. $|V| \times d$.